

actually be, given the information available to the agent. In much the same way, an agent's desires in some state are characterized by a set of states that are consistent with the agent's desires. Intentions are represented similarly. There are several advantages to the possible-worlds model: it is well studied and well understood, and the associated mathematics of correspondence theory is extremely elegant. These attractive features make possible worlds the semantics of choice for almost every researcher in formal agent theory. However, there are also a number of serious drawbacks to possible-worlds semantics. First, possible-worlds semantics imply that agents are logically perfect reasoners (in that their deductive capabilities are sound and complete), and they have infinite resources available for reasoning. No real agent, artificial or otherwise, has these properties.

Second, possible-worlds semantics are generally *ungrounded*. That is, there is usually no precise relationship between the abstract accessibility relations that are used to characterize an agent's state, and any concrete computational model. As we shall see in later sections, this makes it difficult to go from a formal specification of a system in terms of beliefs, desires, and so on, to a concrete computational system. Similarly, given a concrete computational system, there is generally no way to determine what the beliefs, desires, and intentions of that system are. If temporal modal logics such as these are to be taken seriously as *specification* languages, then this is a significant problem.

17.4.2 Logic in implementation

Specification is not (usually!) the end of the story in software development. Once given a specification, we must implement a system that is correct with respect to this specification. The next issue we consider is this move from abstract specification to concrete computational model. There are at least three possibilities for achieving this transformation:

1. Manually refine the specification into an executable form via some principled but informal refinement process (as is the norm in most current software development).
2. Directly execute or animate the abstract specification.
3. Translate or compile the specification into a concrete computational form using an automatic translation technique (cf. the synthesis of agents, discussed in [Chapter 2](#)).

In the subsections that follow, we shall investigate each of these possibilities in turn.

Refinement

At the time of writing, most software developers use structured but informal techniques to transform specifications into concrete implementations. Probably the most common techniques in widespread use are based on the idea of top-down refinement. In this approach, an abstract system specification is *refined* into a number of smaller, less abstract subsystem specifications, which together satisfy the original specification. If these subsystems are still too abstract to be implemented directly, then they are also refined. The process recurses until the derived subsystems are simple enough to be directly implemented. Throughout, we are obliged to demonstrate that each step represents a true refinement of the more abstract specification that preceded it. This demonstration may take the form of a formal proof, if our specification is presented in, say, Z [Spivey, [1992](#)] or VDM [Jones, [1990](#)]. More usually, justification is by informal argument. Object-oriented analysis and design techniques, which also tend to be structured but informal, are also increasingly playing a role in the development of systems (see for example [Booch [1994](#)]).

development of systems (see, for example, [Pnueli, 1982]).

For *functional* systems, which simply compute a function of some input and then terminate, the refinement process is well understood, and comparatively straightforward. Such systems can be specified in terms of preconditions and postconditions (e.g. using Hoare logic [Hoare, 1969]). Refinement calculi exist, which enable the system developer to take a precondition and postcondition specification, and from it systematically derive an implementation through the use of proof rules [Morgan, 1994]. Part of the reason for this comparative simplicity is that there is often an easily understandable relationship between the preconditions and postconditions that characterize an operation and the program structures required to implement it.

For agent systems, which fall into the category of Pnuelian reactive systems (see the discussion in [Chapter 2](#)), refinement is not so straightforward. This is because such systems must be specified in terms of their *ongoing* behaviour – they cannot be specified simply in terms of preconditions and postconditions. In contrast to precondition and postcondition formalisms, it is not so easy to determine what program structures are required to realize such specifications. As a consequence, researchers have only just begun to investigate refinement and design techniques for agent-based systems.

Directly executing agent specifications

One major disadvantage with manual refinement methods is that they introduce the possibility of error. If no proofs are provided, to demonstrate that each refinement step is indeed a true refinement, then the correctness of the implementation process depends upon little more than the intuitions of the developer. This is clearly an undesirable state of affairs for applications in which correctness is a major issue. One possible way of circumventing this problem, which has been widely investigated in mainstream computer science, is to get rid of the refinement process altogether, and *directly execute* the specification.

It might seem that suggesting the direct execution of complex agent specification languages is naive – it is exactly the kind of suggestion that detractors of logic-based AI hate. One should therefore be very careful about what claims or proposals one makes. However, in certain circumstances, the direct execution of agent specification languages *is* possible.

What does it mean, to execute a formula ϕ of logic L ? It means generating a logical model, M , for ϕ , such that $M \models \phi$ [Fisher, 1996]. If this could be done without interference from the environment – if the agent had complete control over its environment – then execution would reduce to constructive theorem-proving, where we show that ϕ is satisfiable by building a model for ϕ . In reality, of course, agents are *not* interference-free: they must iteratively construct a model in the presence of input from the environment. Execution can then be seen as a two-way iterative process:

- environment makes something true
- agent responds by doing something, i.e. making something else true in the model
- environment responds, making something else true
- etc.

Execution of logical languages and theorem-proving are thus closely related. This tells us that the execution of sufficiently rich (quantified) languages is not possible (since any language equal in expressive power to first-order logic is undecidable).

A useful way to think about execution is as if the agent is *playing a game* against the environment. The specification represents the goal of the game: the agent must keep the goal satisfied, while the environment tries to prevent the agent from doing so. The game is played by agent and environment taking turns to build a little more of the model. If the specification ever becomes false in the (partial) model, then the agent loses. In real reactive systems, the game is never over: the agent must continue to play forever. Of course, some specifications (logically inconsistent ones) cannot ever be satisfied. A *winning strategy* for building models from (satisfiable) agent specifications in the presence of arbitrary input from the environment is an execution algorithm for the logic.

Automatic synthesis from agent specifications

An alternative to direct execution is *compilation*. In this scheme, we take our abstract specification, and transform it into a concrete computational model via some automatic synthesis process. The main perceived advantages of compilation over direct execution are in run-time efficiency. Direct execution of an agent specification, as in Concurrent MetateM, described in [Chapter 3](#), typically involves manipulating a symbolic representation of the specification at run-time. This manipulation generally corresponds to reasoning of some form, which is computationally costly (and, in many cases, simply impracticable for systems that must operate in anything like real time). In contrast, compilation approaches aim to reduce abstract symbolic specifications to a much simpler computational model, which requires no symbolic representation. The ‘reasoning’ work is thus done offline, at compile-time; execution of the compiled system can then be done with little or no run-time symbolic reasoning. As a result, execution is much faster. The advantages of compilation over direct execution are thus those of compilation over interpretation in mainstream programming.

Compilation approaches usually depend upon the close relationship between models for temporal/modal logic (which are typically labelled graphs of some kind) and automata-like finite-state machines. Crudely, the idea is to take a specification ϕ , and do a *constructive proof* of the implementability of ϕ , wherein we show that the specification is satisfiable by systematically attempting to build a model for it. If the construction process succeeds, then the specification is satisfiable, and we have a model to prove it. Otherwise, the specification is unsatisfiable. If we have a model, then we ‘read off’ the automaton that implements ϕ from its corresponding model. The most common approach to constructive proof is the *semantic tableaux* method of [Smullyan, [1968](#)].

In mainstream computer science, the compilation approach to automatic program synthesis has been investigated by a number of researchers. Perhaps the closest to our view is the work of [Pnueli and Rosner, [1989](#)] on the automatic synthesis of reactive systems from branching time temporal logic specifications. The goal of their work is to generate reactive systems, which share many of the properties of our agents (the main difference being that reactive systems are not generally required to be capable of rational decision-making in the way we described above). To do this, they specify a reactive system in terms of a first-order branching time temporal logic formula $\forall x \exists y A\phi(x, y)$: the predicate ϕ characterizes the relationship between inputs to the system (x) and outputs (y). Inputs may be thought of as sequences of environment states, and outputs as corresponding sequences of actions. The A is the universal path quantifier. The specification is intended to express the fact that, in all possible futures, the desired relationship ϕ holds between the inputs to the system, x , and its

possible futures, the desired relationship φ holds between the inputs to the system, x , and its outputs, y . The synthesis process itself is rather complex: it involves generating a Rabin tree automaton, and then checking this automaton for emptiness. Pnueli and Rosner show that the time complexity of the synthesis process is double exponential in the size of the specification, i.e. $O(2^{2^{c \cdot n}})$, where c is a constant and $n = |\varphi|$ is the size of the specification φ . The size of the synthesized program (the number of states it contains) is of the same complexity.

The Pnueli–Rosner technique is rather similar to (and in fact depends upon) techniques developed by Wolper, Vardi, and colleagues for synthesizing Büchi automata from linear temporal logic specifications [Vardi and Wolper, 1994]. Büchi automata are those that can recognize ω -regular expressions: regular expressions that may contain infinite repetition. A standard result in temporal logic theory is that a formula ϕ of linear time temporal logic is satisfiable if and only if there exists a Büchi automaton that accepts just the sequences that satisfy ϕ . Intuitively, this is because the sequences over which linear time temporal logic is interpreted can be viewed as ω -regular expressions. This result yields a decision procedure for linear time temporal logic: to determine whether a formula ϕ is satisfiable, construct an automaton that accepts just the (infinite) sequences that correspond to models of ϕ ; if the set of such sequences is empty, then ϕ is unsatisfiable.

Similar automatic synthesis techniques have also been deployed to develop concurrent system skeletons from temporal logic specifications. Manna and Wolper present an algorithm that takes as input a linear time temporal logic specification of the *synchronization* part of a concurrent system, and generates as output a program skeleton (based upon Hoare’s CSP formalism [Hoare, 1978]) that realizes the specification [Manna and Wolper, 1984]. The idea is that the functionality of a concurrent system can generally be divided into two parts: a functional part, which actually performs the required computation in the program, and a synchronization part, which ensures that the system components cooperate in the correct way. For example, the synchronization part will be responsible for any mutual exclusion that is required. The synthesis algorithm (like the synthesis algorithm for Büchi automata, above) is based on Wolper’s tableau proof method for temporal logic [Wolper, 1985]. Very similar work is reported by [Clarke and Emerson, 1981]: they synthesize synchronization skeletons from branching time temporal logic (CTL) specifications.

Perhaps the best-known example of this approach to agent development is the *situated automata* paradigm of [Rosenschein and Kaelbling, 1996], discussed in [Chapter 5](#).

17.4.3 Logic in verification

Once we have developed a concrete system, we need to show that this system is correct with respect to our original specification. This process is known as *verification*, and it is particularly important if we have introduced any informality into the development process. For example, any manual refinement, done without a formal proof of refinement correctness, creates the possibility of a faulty transformation from specification to implementation. Verification is the process of convincing ourselves that the transformation was sound. We can divide approaches to the verification of systems into two broad classes: (1) *axiomatic*, and (2) *semantic* (model checking). In the subsections that follow, we shall look at the way in which these two approaches have evidenced themselves in agent-based systems.

Deductive verification

Axiomatic approaches to program verification were the first to enter the mainstream of computer science, with the work of Hoare in the late 1960s [Hoare, [1969](#)]. Axiomatic verification requires that we can take our concrete program, and from this program systematically derive a logical theory that represents the behaviour of the program. Call this the program theory. If the program theory is expressed in the same logical language as the original specification, then verification reduces to a proof problem: show that the specification is a theorem of (equivalently, is a logical consequence of) the program theory.

The development of a program theory is made feasible by *axiomatizing* the programming language in which the system is implemented. For example, Hoare logic gives us more or less an axiom for every statement type in a simple Pascal-like language. Once given the axiomatization, the program theory can be derived from the program text in a systematic way.

Perhaps the most relevant work from mainstream computer science is the specification and verification of reactive systems using temporal logic, in the way pioneered by Pnueli, Manna, and colleagues (see, for example, [Manna and Pnueli, [1995](#)]). The idea is that the computations of reactive systems are infinite sequences, which correspond to models for linear temporal logic. Temporal logic can be used both to develop a system specification, and to axiomatize a programming language. This axiomatization can then be used to systematically derive the theory of a program from the program text. Both the specification and the program theory will then be encoded in temporal logic, and verification hence becomes a proof problem in temporal logic.

Comparatively little work has been carried out within the agent-based systems community on axiomatizing multiagent environments. I shall review just one approach.

In [Wooldridge, [1992](#)], an axiomatic approach to the verification of multiagent systems was proposed. Essentially, the idea was to use a temporal belief logic to axiomatize the properties of two multiagent programming languages. Given such an axiomatization, a program theory representing the properties of the system could be systematically derived in the way indicated above.

A temporal belief logic was used for two reasons. First, a temporal component was required because, as we observed above, we need to capture the ongoing behaviour of a multiagent system. A belief component was used because the agents that we wish to verify are all symbolic AI systems in their own right. That is, each agent is a symbolic reasoning system, which includes a representation of its environment and desired behaviour. A belief component in the logic allows us to capture the symbolic representations present within each agent.

The two multiagent programming languages that were axiomatized in the temporal belief logic were Shoham's AGENT0 [Shoham, [1993](#)], and Fisher's Concurrent MetateM (see above). The basic approach was as follows.

1. First, a simple abstract model was developed of symbolic AI agents. This model captures the fact that agents are symbolic reasoning systems, capable of communication. The model gives an account of how agents might change state, and what a computation of such a system might look like.
2. The histories traced out in the execution of such a system were used as the semantic basis for a temporal belief logic. This logic allows us to express properties of agents

modelled at stage (1).

3. The temporal belief logic was used to axiomatize the properties of a multiagent programming language. This axiomatization was then used to develop the program theory of a multiagent system.
4. The proof theory of the temporal belief logic was used to verify properties of the system (cf. [Fagin et al., 1995]).

Note that this approach relies on the operation of agents being sufficiently simple that their properties can be axiomatized in the logic. It works for Shoham's AGENT0 and Fisher's Concurrent MetateM largely because these languages have a simple semantics, closely related to rule-based systems, which in turn have a simple logical semantics. For more complex agents, an axiomatization is not so straightforward. Also, capturing the semantics of concurrent execution of agents is not easy (it is, of course, an area of ongoing research in computer science generally).

Model checking

Ultimately, axiomatic verification reduces to a proof problem. Axiomatic approaches to verification are thus inherently limited by the difficulty of this proof problem. Proofs are hard enough, even in classical logic; the addition of temporal and modal connectives to a logic makes the problem considerably harder. For this reason, more efficient approaches to verification have been sought. One particularly successful approach is that of *model checking* [Clarke et al., 2000]. As the name suggests, whereas axiomatic approaches generally rely on syntactic proof, model-checking approaches are based on the semantics of the specification language.

The model-checking problem, in abstract, is quite simple: given a formula ϕ of language L , and a model M for L , determine whether or not ϕ is valid in M , i.e. whether or not $M \models_L \phi$.

Verification by model checking has been studied in connection with temporal logic [Clarke et al., 2000]. The technique once again relies upon the close relationship between models for temporal logic and finite-state machines. Suppose that ϕ is the specification for some system, and P is a program that claims to implement ϕ . Then, to determine whether or not P truly implements ϕ , we proceed as follows:

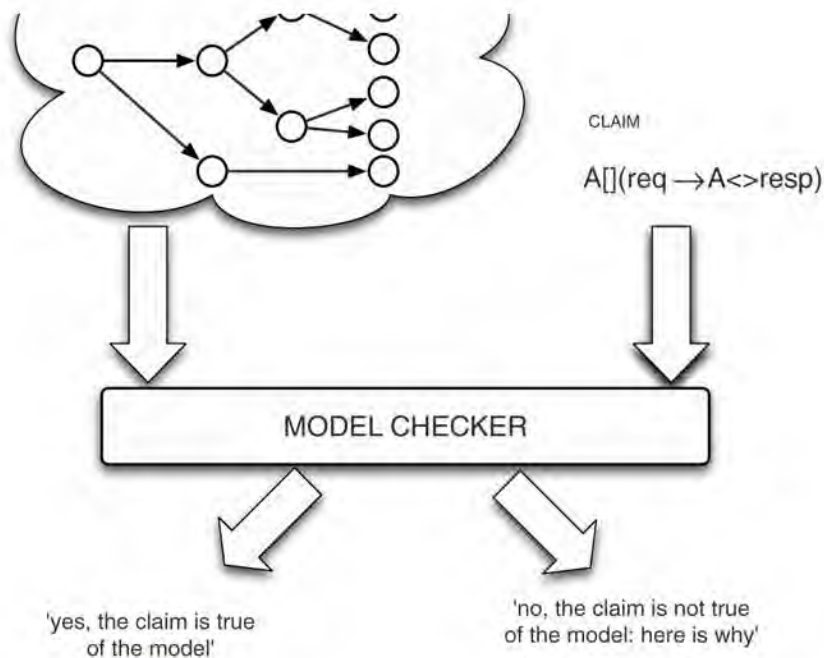
- Take P , and from it generate a model M_P that corresponds to P , in the sense that M_P encodes all the possible computations of P .
- Determine whether or not $M_P \models \phi$, i.e. whether the specification formula ϕ is valid in M_P ; the program P satisfies the specification ϕ just in case the answer is 'yes'.

See [Figure 17.3](#) for an overview.

The main advantage of model checking over axiomatic verification is in complexity: model checking using the branching time temporal logic CTL [Clarke and Emerson, 1981] can be done in time $O(|\phi| \times |M|)$, where $|\phi|$ is the size of the formula to be checked, and $|M|$ is the size of the model against which ϕ is to be checked – the number of states it contains.

Figure 17.3: Model checking.





A number of approaches to model-checking multiagent logics have been developed. [Raimondi and Lomuscio, [2007](#)] describe a model checker called MCMAS. MCMAS allows a user to define a system P using a language closely corresponding to the interpreted systems framework described earlier in this chapter. Claims can be expressed using ATEL, the epistemic extension to the ATL language discussed above. In [Rao and Georgeff, [1993](#)], the authors present an algorithm for model-checking BDI systems. More precisely, they give an algorithm for taking a logical model for their (propositional) BDI logic, and a formula of the language, and determining whether the formula is valid in the model. The technique is closely based on model-checking algorithms for normal modal logics [Clarke et al., [2000](#)]. They show that despite the inclusion of three extra modalities (for beliefs, desires, and intentions) into the CTL branching time framework, the algorithm is still quite efficient, running in polynomial time. So the second step of the two-stage model-checking process described above can still be done efficiently. Similar algorithms have been reported for BDI-like logics in [Benerecetti et al., [1999](#)].

The main problem with model-checking approaches for BDI is that it is not clear how the first step might be realized for BDI logics. Where does the logical model characterizing an agent actually come from? Can it be derived from an arbitrary program P , as in mainstream computer science? To do this, we would need to take a program implemented in, say, Pascal, and from it derive the belief-, desire-, and intention-accessibility relations that are used to give a semantics to the BDI component of the logic. Because, as we noted earlier, there is no clear relationship between the BDI logic and the concrete computational models used to implement agents, it is not clear how such a model could be derived.

Notes and Further Reading

The definitive modern reference to modal logic is [Blackburn et al., [2001](#)]. Written by three of the best people in the field, this is an astonishingly thorough and authoritative work, unlikely to be surpassed for some time to come. The only caveat is that it is emphatically not for the mathematically faint-hearted. For an older (but very readable) introduction to modal logic, see [Chellas, [1980](#)]; an even older, though more wide-ranging, introduction may be found in [Hughes and Cresswell, [1968](#)].

As for the use of modal logics to model knowledge and belief, the definitive work is

[Fagin et al., 1995]. This book, written by the ‘gang of four’, is a joy to read. Clear, detailed, and rigorous, it is (for my money) one of the most important books in the multiagent systems canon. Another useful book, which has perhaps been overshadowed slightly by [Fagin et al., 1995] is [Meyer and van der Hoek, 1995].

Another useful reference to logics of knowledge and belief is [Halpern and Moses, 1992], which includes complexity results and proof procedures. Related work on modelling knowledge has been done by the distributed systems community, who give the worlds in possible-worlds semantics a precise interpretation; for an introduction and further references, see [Halpern, 1987] and [Fagin et al., 1992]. Overviews of formalisms for modelling belief and knowledge may be found in [Fagin et al., 1995; Halpern, 1986; Konolige, 1986; Reichgelt, 1989; Wooldridge, 1992]. A variant on the possible-worlds framework, called the *recursive modelling method*, is described in [Gmytrasiewicz and Durfee, 1993].

Logics which integrate time with mental states are discussed in [Halpern and Vardi, 1989; Kraus and Lehmann, 1988]. Two other important references for temporal aspects are [Shoham, 1988] and [Shoham, 1989]. Thomas developed some logics for representing agent theories as part of her framework for agent programming languages; see [Thomas et al., 1991] and [Thomas, 1993]. For an introduction to temporal logics and related topics, see [Emerson, 1990; Goldblatt, 1987].

Cohen and Levesque’s formalism has been used in an analysis of conflict and cooperation in multiagent dialogue [Galliers, 1988a, b], as well as several studies in the theoretical foundations of cooperative problem solving [Jennings, 1992; Levesque et al., 1990].

With respect to logics that combine different attitudes, perhaps the most important problems still outstanding relate to intention. In particular, the relationship between intention and action has not been formally represented in a satisfactory way. The problem seems to be that having an intention to act makes it more likely that an agent will act, but does not generally guarantee it. While it seems straightforward to build systems that appear to have intentions, it seems much harder to capture this relationship formally. Other problems that have not yet really been addressed in the literature include the management of multiple, possibly conflicting, intentions, and the formation, scheduling, and reconsideration of intentions.

The question of exactly *which combination* of attitudes is required to characterize an agent is also the subject of some debate. As we observed above, a currently popular approach is to use a combination of beliefs, desires, and intentions (hence BDI architectures [Rao and Georgeff, 1991b]). However, there are alternatives: Shoham, for example, suggests that the notion of choice is more fundamental [Shoham, 1990]. Comparatively little work has yet been done on formally comparing the suitability of these various combinations. One might draw a parallel with the use of temporal logics in mainstream computer science, where the expressiveness of specification languages is by now a well-understood research area [Emerson, 1990]. Perhaps the obvious requirement for the short term is experimentation with real agent specifications, in order to gain a better understanding of the relative merits of different formalisms.

More generally, the kinds of logics used in agent theory tend to be rather elaborate, typically containing many modalities which interact with each other in subtle ways. The definitive reference to multimodal logics is [Coburn et al., 2002] (he says that this is a

definitive reference to multimodal logics is [Gabbay et al., 2005] (be aware that this is a deeply technical area).

Finally, there is often some confusion about the role played by a theory of agency. The view we take is that such theories represent *specifications* for agents. The advantage of treating agent theories as specifications, and agent logics as specification languages, is that the problems and issues we then face are familiar from the discipline of software engineering. How useful or expressive is the specification language? How concise are agent specifications? How does one refine or otherwise transform a specification into an implementation? However, the view of agent theories as specifications is not shared by all researchers. Some intend their agent theories to be used as knowledge representation formalisms, which raises the difficult problem of algorithms to reason with such theories. Still others intend their work to formalize a concept of interest in cognitive science or philosophy (this is, of course, what Hintikka intended in his early work on logics of knowledge of belief). What is clear is that it is important to be precise about the role one expects an agent theory to play.

An informal discussion of intention may be found in [Bratman, 1987], or more briefly in [Bratman, 1990]. Further work on modelling intention may be found in [Grosz and Sidner, 1990; Konolige and Pollack, 1993; Sadek, 1992]. A critique of Cohen and Levesque's theory of intention is presented in [Singh, 1992].

Although I have not discussed formalisms for reasoning about action here, we suggested above that an agent logic would need to incorporate some mechanism for representing agents' actions. Our reason for avoiding the topic is simply that the field is so big that it deserves a whole review in its own right. Good starting points for AI treatments of action are [Allen, 1984; Allen et al., 1990, 1991]. Other treatments of action in agent logics are based on formalisms borrowed from mainstream computer science, notably dynamic logic (originally developed to reason about computer programs) [Harel et al., 2000]. The logic of *seeing to it that* has been discussed in the formal philosophy literature [Belnap, 1991; Belnap and Perloff, 1988; Perloff, 1991; Segerberg, 1989].

See [Wooldridge, 1997] for a discussion on the possibility of using logic to engineer agent-based systems.

With respect to the possibility of directly executing agent specifications, a number of problems suggest themselves. The first is that of finding a concrete computational interpretation for the agent specification language in question. To see what we mean by this, consider models for the agent specification language in Concurrent MetateM. These are very simple: essentially just linear discrete sequences of states. Temporal logic is (among other things) simply a language for expressing *constraints* that must hold between successive states. Execution in Concurrent MetateM is thus a process of generating constraints as past-time antecedents are satisfied, and then trying to build a next state that satisfies these constraints. Constraints are expressed in temporal logic, which implies that they may only be in certain, regular forms. Because of this, it is possible to devise an algorithm that is guaranteed to build a next state if it is possible to do so. Such an algorithm is described in [Barringer et al., 1989]. The agent specification language upon which Concurrent MetateM is based thus has a concrete computational model, and a comparatively simple execution algorithm. Contrast this state of affairs with languages like that of [Cohen and Levesque, 1990a], where we have not only a temporal dimension

to the logic, but also modalities for referring to beliefs, desires, and so on. In general, models for these logics have *ungrounded* semantics. That is, the semantic structures that underpin these logics (typically accessibility relations for each of the modal operators) have no concrete computational interpretation. As a result, it is not clear how such agent specification languages might be executed. Another obvious problem is that execution techniques based on theorem-proving are inherently limited when applied to sufficiently expressive (first-order) languages, as first-order logic is undecidable. However, complexity is a problem even in the propositional case. For ‘vanilla’ propositional logic, the decision problem for satisfiability is NP-complete [Fagin et al., 1995, p. 72]; richer logics, of course, have more complex decision problems.

Despite these problems, the undoubted attractions of direct execution have led to a number of attempts to devise executable logic-based agent languages. Rao proposed an executable subset of BDI logic in his AgentSpeak(L) language [Rao, 1996a]. Building on this work, Hindriks and colleagues developed the 3APL agent programming language [Hindriks et al., 1998, 1999]. Lespérance, Reiter, Levesque, and colleagues developed the Golog language throughout the latter half of the 1990s as an executable subset of the situation calculus [Lésperance et al., 1996; Levesque et al., 1996]. Fagin and colleagues have proposed *knowledge-based programs* as a paradigm for executing logical formulae which contain epistemic modalities [Fagin et al., 1995, 1997]. Although considerable work has been carried out on the properties of knowledge-based programs, comparatively little research to date has addressed the problem of how such programs might actually be executed.

Turning to automatic synthesis, the techniques described above have been developed primarily for propositional specification languages. If we attempt to extend these techniques to more expressive, first-order specification languages, then we again find ourselves coming up against the undecidability of quantified logic. Even in the propositional case, the theoretical complexity of theorem-proving for modal and temporal logics is likely to limit the effectiveness of compilation techniques: given an agent specification of size 1000, a synthesis algorithm that runs in exponential time when used offline is no more useful than an execution algorithm that runs in exponential time online. [Kupferman and Vardi, 1997] is a recent article on automatic synthesis from temporal logic specifications.

Another problem with respect to synthesis techniques is that they typically result in finite-state, automata-like machines, which are less powerful than Turing machines. In particular, the systems generated by the processes outlined above cannot modify their behaviour at run-time. In short, they cannot learn. While for many applications this is acceptable – even desirable – for equally many others, it is not. In expert assistant agents, of the type described in [Maes, 1994a], learning is pretty much the *raison d’être*. Attempts to address this issue are described in [Kaelbling, 1993].

Turning to verification, axiomatic approaches suffer from two main problems. First, the temporal verification of reactive systems relies upon a simple model of concurrency, where the actions that programs perform are assumed to be atomic. We cannot make this assumption when we move from programs to agents. The actions we think of agents as performing will generally be much more coarse-grained. As a result, we need a more realistic model of concurrency. The second problem is the difficulty of the proof problem

for agent specification languages. The theoretical complexity of proof for many of these logics is quite daunting.

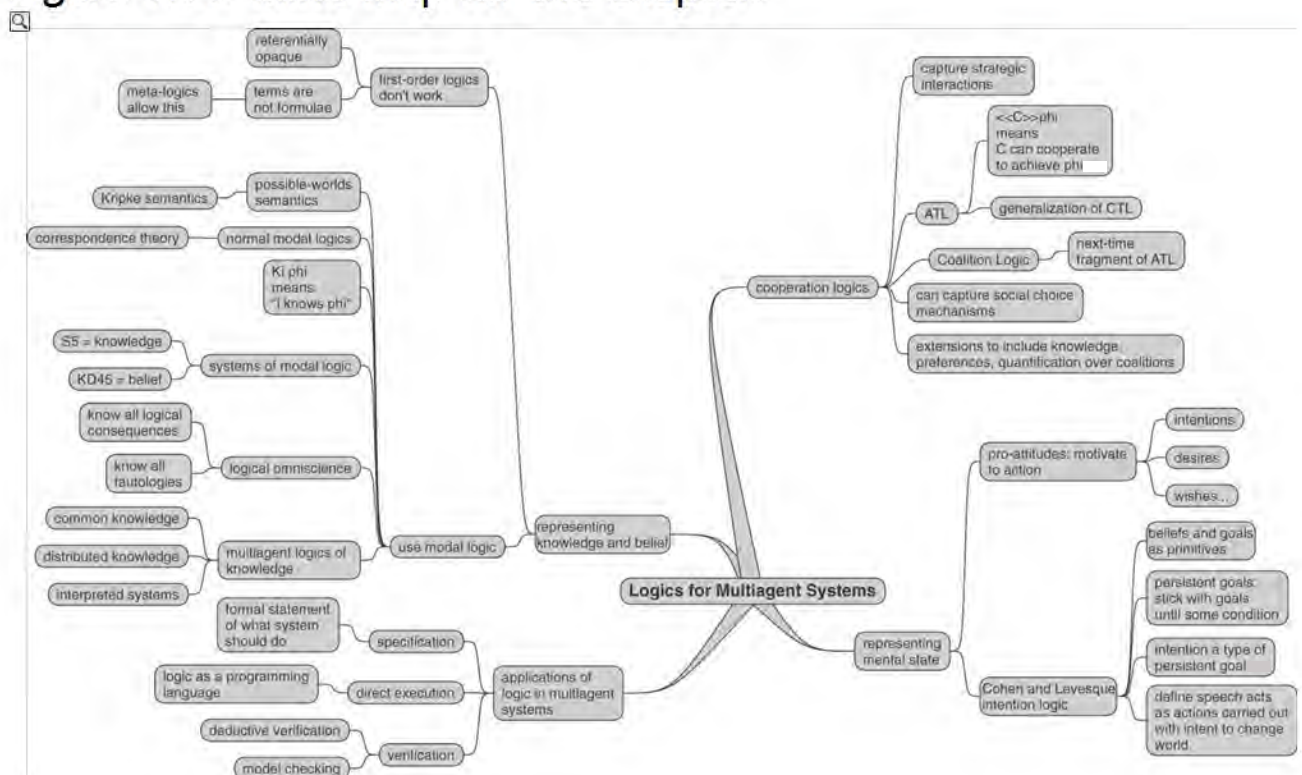
Hindriks and colleagues have used Plotkin's structured operational semantics to axiomatize their 3APL language [Hindriks et al., [1998](#), [1999](#)].

With respect to model-checking approaches, the main problem, as we indicated above, is again the issue of ungrounded semantics for agent specification languages. If we cannot take an arbitrary program and say, for this program, what its beliefs, desires, and intentions are, then it is not clear how we might verify that this program satisfied a specification expressed in terms of such constructs.

Formalisms for reasoning about agents have come a long way since Hintikka's pioneering work on logics of knowledge and belief [Hintikka, [1962](#)]. Within AI, perhaps the main emphasis of subsequent work has been on attempting to develop formalisms that capture the relationship between the various elements that comprise an agent's cognitive state; the paradigm example of this work is the well-known theory of intention developed by [Cohen and Levesque, [1990a](#)]. Despite the very real progress that has been made, there still remain many fairly fundamental problems and issues outstanding.

Class reading: [Rao and Georgeff, [1992](#)]. This paper is not too formal, but is focused on the issue of when a particular agent implementation can be said to implement a particular theory of agency.

Figure 17.4: Mind map for this chapter.



Back Matter

[Appendix A A History Lesson](#)

[Appendix B Afterword](#)

[Glossary of Key Terms](#)

[References](#)

[Index](#)

We often naively assume that technologies and academic disciplines somehow spontaneously emerge from nowhere, fully formed and well defined. Of course, nothing could be further from the truth. They tend to emerge in a rather haphazard fashion, and are shaped as much as anything by the personalities, prejudices, and fashions of the time. The multiagent systems field is no exception to this rule. Indeed, the number of other disciplines that have contributed to the multiagent systems field is so large that the story is even murkier than is normally the case. In this section, therefore, I will attempt to give a potted history of the field, identifying some of the milestones and key players.

Please note: more than any other part of the book, this appendix is subjective. The interpretation of events is my own, and as I was not personally present for many of the events described, I have sometimes had to construct a (semi)coherent history from the literature. It follows that not everyone will agree with my version of events. I welcome comments and suggestions, which I will endeavour to take into account in the event of a third edition.

A history of intelligent agent research

We could spend a month debating exactly when the multiagent systems field was born; as with the computing and artificial intelligence fields, we could identify many agent-related ideas that emerged prior to the 20th century. But we can say with some certainty that the agents field (although not necessarily the *multiagent* systems field) was alive following the now famous 1956 Dartmouth workshop at which John McCarthy coined the term ‘artificial intelligence’. The notion of an ‘agent’ is clearly evident in the early AI literature. For example, consider the Turing test, put forward by Alan Turing as a way of settling the argument about whether machines could ever be considered to be intelligent. The idea of the test is that a person interrogates some entity via a monitor. The person is free to put forward any questions or statements whatsoever, and after five minutes is required to decide whether the entity at the other end is another person or a machine. If such a test cannot distinguish a particular program from a person, then, Turing argued, the program must be considered intelligent to all intents and purposes. Clearly, we can think of the program at the other end of the teletype as an agent – the program is required to respond, in real time, to statements made by the person, and the rules of the test prohibit interference with the program. It exhibits some autonomy, in other words.

Although the idea of an agent was clearly present in the early days of AI, there was comparatively little development in the idea of agents as *holistic* entities (i.e. integrated systems capable of independent autonomous action) until the mid 1980s; below, we will see why this happened.

Between about 1969 and 1985, research into systems capable of independent action was carried out primarily within the AI planning community, and was dominated by what I will call the ‘reasoning and planning’ paradigm. AI planning (see [Chapter 3](#)) is essentially automatic programming: a planning algorithm takes as input a description of a goal to be achieved, a description of how the world currently is, and a description of a number of available actions and their effects. The algorithm then outputs a plan – essentially, a program – which describes how the available actions can be executed so as to bring about the desired goal. The best-known, and most influential, planning algorithm was the STRIPS system [Fikes and Nilsson, [1971](#)]. STRIPS was so influential for several reasons. First, it developed a particular notation for describing actions and their effects that remains to this day the foundation for most

action representation notations. Second, it emphasized the use of formal, logic-based notations for representing both the properties of the world and the actions available. Finally, STRIPS was actually used in an autonomous robot called Shakey at Stanford Research Institute.

The period between the development of STRIPS and the mid 1980s might be thought of as the ‘classic’ period in AI planning. There was a great deal of progress in developing planning algorithms, and understanding the requirements for representation formalisms for the world and actions. At the risk of over generalizing, this work can be characterized by two features, both of which were pioneered in the STRIPS system:

- the use of explicit, symbolic representations of the world
- an increasing emphasis on the use of *formal*, typically logic-based representations, and, associated with this work, an increasing emphasis on *deductive* decision-making (i.e. decision-making as logical proof).

Rodney Brooks recalls the title of a seminar series in the early 1980s: *From Pixels to Predicates*, which for him neatly summed up the spirit of the age [Brooks, [1999](#), p. ix]. By the mid 1980s, however, some researchers were having doubts about the assumptions on which this work was based, and were beginning to voice concerns about the direction in which research on the design of agents was going.

I noted above that the idea of ‘integrated’ or ‘whole’ agents, as opposed to agent behaviours (such as learning and planning) did not greatly evolve between the emergence of AI and the mid 1980s. The reason for this lack of progress is as follows. In the early days of AI, there was a great deal of scepticism about computers being able to exhibit ‘intelligent’ behaviour. A common form of argument was that ‘computers will never be able to X ’, where X = solve problems, learn, communicate in natural language, and so on (see [Russell and Norvig, [1995](#), p. 823] for a discussion). A natural response to these kinds of arguments by those interested in AI was to build systems that could exhibit behaviour X . These early systems that could plan, learn, communicate in natural language, and so on, led to the emergence of a number of subdisciplines in AI: the planning, learning, and natural language communication communities, for example, all have their own conferences, workshops, and literature. And all these communities evolved from the groundbreaking work done on these types of behaviour in the early days of AI.

But, critically, there were few attempts to actually *integrate* these kinds of behaviours into whole systems – agents. Instead, researchers focused on building better planning algorithms, better learning algorithms, and so on. By and large, they did not address the problem of how such algorithms might be placed in the context of a ‘whole’ agent. As a consequence, by the mid 1980s (as we will see below), significant progress had been made in each of these component disciplines, but there was a dearth of experience with respect to building agents from these components. Worse, some researchers began to argue that, because no consideration had been given to how these components might be integrated to build an agent, the component technologies had evolved in such a way that the integration and use of these components in realistic systems was, for all practical purposes, impossible: most component techniques had been evaluated on problems that were some distance from being as complex as real-world problems.

The upshot of all this was that, some researchers argued, ‘vertical decomposition’ of an agent into the different functional components was based on the flawed assumption that the

components could be easily integrated to produce an agent.

In addition, it was argued that ‘Artificial intelligence research has foundered on the issue of representation’ [Brooks, [1991b](#)].¹ The problems associated with building an agent that decides what to do by manipulating a symbolic (particularly logic-based) representation of the world were simply too deep to make the approach viable. The conclusion that many researchers came to was that a completely new approach was required.

The result was an entirely new approach to building agents, variously referred to as ‘behavioural AI’, ‘reactive AI’, or simply ‘the new AI’. Rodney Brooks was perhaps the most vocal member of this community, and came to be seen as the champion of the movement. The workers in this area were not united by any common approaches, but certain themes did recur in this work. Recurring themes were the rejection of architectures based on symbolic representations, an emphasis on a closer coupling between the agent’s environment and the action it performs, and the idea that intelligent behaviour can be seen to emerge from the interaction of a number of much simpler behaviours.

The term ‘furore’ might reasonably be used to describe the response from the symbolic and logical reasoning communities to the emergence of behavioural AI. Some researchers seemed to feel that behavioural AI was a direct challenge to the beliefs and assumptions that had shaped their entire academic careers. Not surprisingly, they were not predisposed simply to abandon their ideas and research programs.

I do not believe that there was (or is) a clear-cut ‘right’ or ‘wrong’ in this debate. With the benefit of hindsight, it seems clear that much symbolic AI research had wandered into the realms of abstract theory, and did not connect in any realistic way with the reality of building and deploying agents in realistic scenarios. It also seems clear that the decomposition of AI into components such as planning and learning, without any emphasis on synthesizing these components into an integrated architecture, was perhaps not the best strategy for AI as a discipline. But it also seems that some claims made by members of the behavioural community were extreme, and in many cases suffered from the over-optimism that AI itself suffered from in its early days.

The practical implications of all this were threefold.

- The behavioural AI community to a certain extent split off from the mainstream AI community. Taking inspiration from biological metaphors, many of the researchers in behavioural AI began working in a community that is today known as ‘artificial life’ (alife).
- Mainstream AI began to recognize the importance of integrating the components of intelligent behaviour into agents, and, from the mid 1980s to the present day, the area of agent architectures has grown steadily in importance.
- Within AI, the value of testing and deploying agents in *realistic* scenarios (as opposed to simple, contrived, obviously unrealistic scenarios) was recognized. This led to the emergence of such scenarios as the RoboCup robotic soccer challenge, in which the aim was to build agents that could actually play a game of soccer against a team of robotic opponents [RoboCup, [2001](#)].

So, by the mid 1980s, the area of *agent architectures* was becoming established as a specific research area within AI itself.

Most researchers in the agent community accept that neither a purely logicist or reasoning approach nor a purely behavioural approach is the best route to building agents capable of intelligent autonomous action. Intelligent autonomous action seems to imply the capability for *both* reasoning and reactive behaviour. As Innes Ferguson succinctly put it [Ferguson, [1992a](#), p. 31]:

It is both desirable and feasible to combine suitably designed deliberative and non-deliberative control functions to obtain effective, robust, and flexible behaviour from autonomous, task-achieving agents operating in complex environments.

This recognition led to the development of a range of *hybrid* architectures, which attempt to combine elements of both behavioural and deliberative systems. At the time of writing, hybrid approaches dominate in the literature.

A history of multiagent systems research

Research in *multiagent* systems progressed quite independently of research into individual agents until about the early 1990s. It is interesting to note that although the notion of an agent as an isolated system was evident in the early AI literature, the notion of a *multiagent system* did not begin to gain prominence until the early 1980s. Some attention was certainly given to interaction between artificial agents and humans, in the form of research on natural language understanding and generation. (The Turing test, after all, is predicated on the development of other computer systems with such abilities.) But almost no consideration was given to interactions among artificial agents.

To understand how multiagent systems research emerged, it is necessary to go back to the work of Alan Newell and Herb Simon on *production systems* [Russell and Norvig, [1995](#), pp. 297–298]. A production system is essentially a collection of ‘pattern → action’ rules, together with a *working memory* of facts. The production system works by *forward chaining* through the rules, continually matching the left-hand side of rules against working memory, and performing the action of a rule that fires. The action may involve adding or removing facts from working memory. A key problem with standard production systems is that the system’s knowledge is *unstructured*: all the rules are collected together into a single amorphous set. This can make it hard to understand and structure the behaviour of the production system. The need for structured knowledge led to the earliest work that was recognizably multiagent systems: *blackboard systems* [Engelmore and Morgan, [1988](#)]. A blackboard system is characterized by two main attributes (see [Figure A.1](#)):

- a collection of independent entities known as *knowledge sources*, each of which has some specialized knowledge, typically encoded in the form of rules
- a shared data structure known as a *blackboard*, which knowledge sources use to communicate.

Knowledge sources in blackboard systems are capable of reading and writing to the blackboard data structure, and problem solving proceeds by the knowledge sources each monitoring the blackboard and writing to it when they can contribute partial problem solutions. The blackboard metaphor was neatly described by Alan Newell long before the blackboard model became widely known:

Metaphorically we can think of a set of workers, all looking at the same blackboard: each

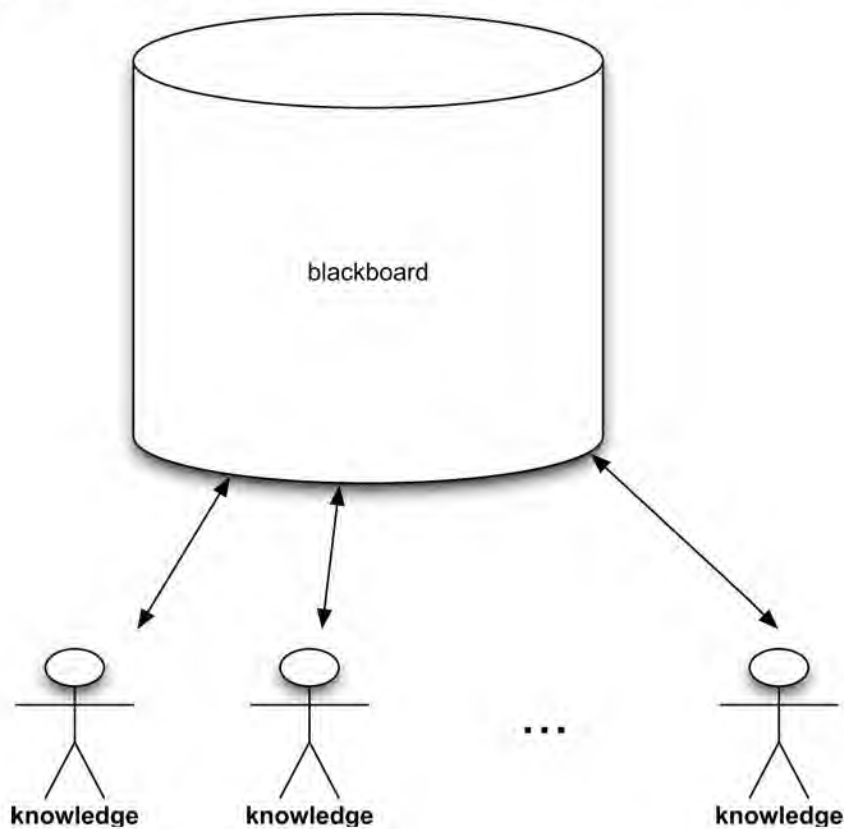
is able to read everything that is on it, and to judge when he has something worthwhile to add to it. This conception is ... a set of demons, each independently looking at the total situation and shrieking in proportion to what they see fits their natures.

[Newell, 1962]

(quoted from [Engelmore and Morgan, 1988, p. 16])

The first, and probably best-known, blackboard system was the Hearsay system for speech understanding, developed in the early 1970s under the supervision of [Reddy et al., 1973]. One of Reddy's co-workers on the Hearsay project was Victor ('Vic') Lesser, who moved to the University of Massachusetts at Amherst in 1977. Lesser had worked on multiprocessing computer systems in the 1960s, and was well aware of the potential value of *parallelism*. He recognized that the blackboard model, with its multiple knowledge sources each contributing partial solutions to the overall problem, provided a natural metaphor for problem solving that exploited parallelism [Fennell and Lesser, 1977]. But the blackboard model is essentially a shared data structure architecture; in order for knowledge sources to communicate in a classical blackboard architecture, they need to write to this shared data structure, which implies that there can only ever be a single thread of control writing to the blackboard at any given time. This limits the parallelism possible in the classical blackboard model, as the shared data structure (and the need to synchronize access to this data structure) introduces a bottleneck in problem solving. The next step was to introduce 'true' parallelism into the architecture, by allowing *multiple* blackboard systems to communicate by message passing: Lesser and Erman did this in the late 1970s, still working within the Hearsay problem domain of speech understanding [Lesser and Erman, 1980].

Figure A.1: A blackboard architecture: a number of knowledge sources encapsulate knowledge about a problem, and communicate by reading and writing on a shared data structure known as a blackboard.



Blackboard systems were highly influential in the early days of multiagent systems, but are no longer a major area of research activity. The definitive reference is [Engelmore and Morgan, 1988], and other useful references include [Erman et al., 1980], [Hayes-Roth, 1985] and [Corkill et al., 1987].

At about the same time as Lesser and colleagues were beginning to build parallel blackboard systems, Doug Lenat proposed the Beings model of problem solving [Lenat, 1975]. This model of problem solving is very similar to the blackboard model, the metaphor being that of a number of experts cooperating to solve problems by asking and answering questions. The beings in Lenat's system were not autonomous agents – they were more closely related to knowledge sources in the blackboard model – but the metaphors of cooperation and distribution are clearly evident.

Throughout the 1970s, several other researchers developed prototypical multiagent systems. The first was Carl Hewitt, who proposed the Actor model of computation. Hewitt obtained his PhD from MIT in 1971 for his work on a system called Planner [Hewitt, 1971]. This system made use of data structures called schemata, which somewhat resemble the knowledge sources in blackboard architectures. After his PhD work, Hewitt was made aware of work on the Smalltalk system under way at Xerox Palo Alto Research Center (Xerox PARC) [Goldberg, 1984]. Smalltalk is widely recognized as the first real object-oriented programming language. Smalltalk made liberal use of the metaphor of *message passing* to describe how objects communicated with one another.² Taking inspiration from Smalltalk, Hewitt proposed the Actor model, in which computation itself is viewed primarily as message passing. The Actor model was described as early as 1973 [Hewitt, 1973], but the best-known and most widely cited expression of the Actor model was a 1977 article published in *Artificial Intelligence* journal [Hewitt, 1977]. The Actor model was Hewitt's expression of some radical views about the future direction of computation, and with the benefit of hindsight, it is remarkable just how far ahead of his time Hewitt was. He recognized that the future of computing itself was inexorably tied to distributed, *open* systems [Hewitt, 1985], and that traditional models of computation were not well suited for modelling or understanding such distributed computation. The Actor paradigm was his attempt to develop a model of computation that more accurately reflected the direction in which computer science was going.

An Actor is a computational system with the following properties.

- Actors are social – they are able to send messages to other Actors.
- Actors are reactive – they carry out computation in response to a message received from another Actor. (Actors are thus *message driven*.)

Intuitively, an Actor can be considered as consisting of

- a *mail address* which names the Actor
- a *behaviour* which specifies what the Actor will do upon receipt of a message.

The possible components of an Actor's behaviour are:

- sending messages to itself or other Actors
- creating more Actors;
- specifying a replacement behaviour

- specifying a replacement behaviour.

Intuitively, the way an Actor works is quite simple:

- upon receipt of a message, the message is matched against the Actor's behaviour (script)

Figure A.2: An Actor for computing factorials.

```

1.  Rec-Factorial with acquaintances self
2.  let communication be an integer n and a customer u
3.  become Rec-Factorial
4.  if n=0
5.      then
6.          send [1] to customer
7.      else
8.          let c=Rec-Customer with acquaintances n and u
9.          {send [n-1, mail address of c] to self}
10. Rec-Customer with acquaintances integer n and customer u
12. let communication be an integer k
13. {send [n * k] to u}

```

- upon a match, the corresponding action is executed, which may involve sending more messages, creating more Actors, or replacing the Actor by another.

An example Actor, which computes the factorial of its argument, is shown in [Figure A.2](#) (from [Agha, 1986]). Receipt of a message containing a non-zero integer n by `Factorial` will result in the following behaviour:

- create an Actor whose behaviour will be to multiply n by an integer it receives and send the reply to the mail address to which the factorial of n was to be sent
- send itself the 'request' to evaluate the factorial of $n-1$ and send the value to the customer it created.

The creation of Actors in this example mirrors the recursive procedures for computing factorials in more conventional programming languages.

The Actor paradigm greatly influenced work on concurrent object languages [Agha et al., 1993]. Particularly strong communities working on concurrent object languages emerged in France (led by Jacques Ferber and colleagues [Ferber and Carle, 1991]) and Japan (led by Akinora Yonezawa, Mario Tokoro, and colleagues [Yonezawa, 1990; Yonezawa and Tokoro, 1997]).

In the late 1970s at Stanford University in California, a doctoral student called Reid Smith was completing his PhD on a system called the Contract Net, in which a number of agents ('problem-solving nodes' in Smith's parlance) solved problems by delegating subproblems to other agents [Smith, 1977, 1980a]. As the name suggests, the key metaphor is that of subcontracting in human organizations. The Contract Net remains to this day one of the most influential multiagent systems developed. It introduced several key concepts into the multiagent systems literature, including the economics metaphor and the negotiation metaphor.

Smith's thesis was published in 1980, a year also notable for the emergence of the first academic forum for research specifically devoted to the new paradigm of multiagent systems. Randy Davis from MIT organized the first workshop on what was then called 'distributed artificial intelligence' (DAI) [Davis, 1980]. Throughout the 1980s, the DAI workshops, held more or less annually in the USA, became the main focus of activity for the new community.

The 1985 workshop, organized by Michael Genesereth and Matt Ginsberg of Stanford University, was particularly important as the proceedings were published as the first real book on the field: the ‘green book’, edited by Michael Huhns [Huhns, [1987](#)]. The proceedings of the 1988 workshop, held at Lake Arrowhead in California, were published two years later as the ‘second green book’ [Gasser and Huhns, [1989](#)]. Another key publication at this time was Bond and Gasser’s 1988 collection *Readings in Distributed Artificial Intelligence* [Bond and Gasser, [1988](#)]. This volume brought together many of the key papers of the field. It was prefaced with a detailed and insightful survey article, which attempted to summarize the key problems and issues facing the field; the survey remains relevant even at the time of writing.

Until about the mid 1980s the emphasis was on ‘parallelism in problem solving’, or *distributed problem solving* as it became known. In other words, the main type of issue being addressed was ‘given a problem, how can we exploit multiple processor architectures to solve this problem?’. In the mid 1980s, a Stanford University PhD student called Jeff Rosenschein fundamentally changed the emphasis of the field, by recognizing that distributed problem-solving systems implicitly assumed ‘common interest’ among the agents in the system. He realized that while such systems are undoubtedly important, they represent a special case of a much more general class of systems, in which agents are assumed to be *self-interested*. In his 1985 paper *Deals Among Rational Agents*, he coined the term ‘benevolent agent’ to describe agents that could be assumed to help out wherever asked [Rosenstein and Genesereth, [1985](#)]. As well as making the critically important distinction between self-interested and benevolent agents, this paper is also significant for a second reason: it was the first paper to make use of techniques from *game theory* to analyse interactions among artificial agents.

The mid 1980s also saw the development of the first general-purpose testbed for experimentation with agent systems. The MACE system (an acronym of ‘multiagent computing environment’) was developed under the supervision of Les Gasser at the University of Southern California [Gasser et al., [1987a](#)]. MACE provided many features that have subsequently become standard in multiagent systems; for example, it pioneered the provision of *acquaintance models* by which agents could have representations of the capabilities and plans of their peers.

Somewhat surprisingly, those active in the field at the time report that interest in DAI actually *waned* throughout the latter half of the 1980s. The reasons for this are unclear; it may well be that this reduction of interest was simply a result of some of the key figures from the early days of the field moving into new jobs and new areas of work. But the seeds sown with the establishment of a regular workshop and the publication of three key books led to an international flowering of interest in DAI. In the late 1980s, the European Commission funded a research project entitled ARCHON (‘architecture for cooperating heterogeneous online systems’), which was originally focused on the problem of getting a number of distinct ‘expert systems’ to pool their expertise in solving problems and diagnosing faults in several industrial domains [Jennings et al., [1996a](#); Perriolat et al., [1996](#); Wittig, [1992](#)]. ARCHON was a *large* project (14 partners across 9 European countries!), and subsequently became recognized as one of the first real industrial applications of agent systems.

At about the same time, the European Commission also funded the MAGMA project (loosely derived from ‘modelling an autonomous agent in a multiagent world’). As part of this project, the participants decided to organize a workshop with the same name; it was held in Cambridge,

UK, in 1989, and was so successful that the MAAMAW workshops became an annual event. Through the early 1990s, led by Yves Demazeau, MAAMAW was the main European forum for agent research [Demazeau and Müller, [1990](#)]. In Japan, the MACC workshops were also established as a regional forum for agent research.

Interest in agent systems grew very rapidly in the first half of the 1990s. There were several reasons for this. The first, and probably most important, reason was the spread of the Internet, which through the 1990s changed from being a tool unknown outside academia to something in everyday use for commerce and leisure across the globe. In many ways, 1994 seems to have been a milestone year for agents. The first reason is that 1994 was the year that the Web emerged; the Mosaic browser only began to reach a truly wide audience in 1994 [Berners-Lee, [1999](#)]. The Web provided an easy-to-use front end for the Internet, enabling people with very limited IT training to productively use the Internet for the first time. The explosive growth of the Internet was perhaps the most vivid illustration possible that the future of computing lay in distributed, networked systems, and that in order to exploit the potential of such distributed systems, new models of computation were required. By the summer of 1994 it was becoming clear that the Internet would be a major proving ground for agent technology (perhaps even the ‘killer application’), although the full extent of this interest was not yet apparent.

As well as the emergence of the Web, 1994 saw the publication in July of a special issue of *Communications of the ACM* that was devoted to intelligent agents. CACM is one of the best-known publications in the computing world, and ACM is arguably its foremost professional body; the publication of a special issue of CACM on agents was therefore some kind of recognition by the computing world that agents were worth knowing about. Many of the articles in this special issue described a new type of agent system, one that acted as a kind of ‘expert assistant’ to a user working with a particular class of application. The vision of agents as intelligent assistants was perhaps articulated most clearly by Pattie Maes from MIT’s Media Lab, who described a number of prototype systems to realize the vision [Maes, [1994a](#)]. Such user interface agents rapidly caught the imagination of a wider community, and, in particular, the commercial possibilities of such technologies were self-evident. A number of agent startup companies were founded to commercialize this technology, including Pattie Maes’s company FireFly (subsequently sold to Microsoft), and Oren Etzioni’s company NetBot (subsequently sold to the Web portal company Excite).

With the growth of the Internet in the late 1990s came electronic commerce (e-commerce), and the rapid international expansion of ‘dot com’ companies. It was quickly realized that e-commerce represented a natural – and potentially very lucrative – application domain for multiagent systems. The idea is that agents can partially automate many of the stages of electronic commerce, from finding a product to buy, through to actually negotiating the terms of agreement [Noriega and Sierra, [1999](#)]. This area of *agent-mediated electronic commerce* became perhaps the largest single application area for agent technology by the turn of the century, and gave an enormous impetus (commercial, as well as scientific) to the areas of *negotiation* and *auctions* in agent systems.

The emergence of agents on and for the Internet gave rise to a new, associated software technology, somewhat distinct from the ‘mainstream’ of agent research and development. In the summer of 1994, a California-based company called General Magic was creating intense interest in the idea of *mobile agents* – programs that could transmit themselves across an electronic network and recommence execution at a remote site [White, [1997](#)]. At the time,

General Magic was distributing a widely read white paper that described Telescript – a programming language intended to realize the vision of mobile agents [White, 1994]. In the event, it was not Telescript but another programming language that caught the imagination of the Internet community: Java. When Netscape incorporated a Java virtual machine into their Navigator browser, they made it possible for browsers to download and execute small programs called *applets*. Applets transformed the Web from being a large but essentially static collection of linked documents to being an active and dynamic system of interworking components. The potential to the computer science community was obvious and this gave Java an enormous impetus, as a way of animating the Internet, but also as a powerful, well-designed object-oriented programming language in its own right. Although they are not agents in the sense that I use the term in this book, applets give a hint of what might be possible in the future.

A number of mobile agent frameworks were rapidly developed and released as Java packages, and interest in Telescript waned. At the time of writing, Java is the programming language of choice not just for agent systems, but also, it seems, for most other applications in computing. Java was never intended to be an ‘agent programming language’ (although it can of course be used for programming agents), but since it was first released, the language has been progressively extended to give it ever more agent-like features. A good example is the JINI framework, which allows objects to advertise their services and, in a simple way, to cooperate with one another in a similar way to that proposed by the agent community [Oaks and Wong, 2000].

By the mid 1990s, the level of industrial interest in agent systems was such that *standardization* became a major issue, and some researchers began to suspect that the lack of recognized international standards was an impediment to the wider take-up of agent technology. The early 1990s had already seen some activity in this area, in the form of the US-based Knowledge Sharing Effort, within which two influential languages for agent communication were developed: KQML and KIF [Patil et al., 1992]. However, these languages were never formally standardized, which led to great difficulties for agent developers who actually wanted to use them in open settings. As a result, in 1995, the FIPA movement began its work on standards for agent systems [FIPA, 2001]. The centrepiece of the FIPA initiative was a language for agent communication. By the end of the decade, many major IT and telecommunications companies had become involved in the FIPA movement, and a set of prototypical standards had been developed. At the time of writing, the major initiative under way is to *deploy* these languages in real settings, and – hopefully – demonstrate their value to agent developers.

Another body of work that arose in the mid 1990s, led by Rosaria Conte, Jim Doran, and Nigel Gilbert, was the use of multiagent systems for modelling natural societies [Gilbert and Conte, 1995; Gilbert and Doran, 1994]. The Simulating Societies (SimSoc) workshop, first held in 1993, brought together researchers who were interested in using multiagent systems to gain insights into the behaviour of human societies.

Finally, in the late 1990s, researchers in multiagent systems began to seek increasingly realistic domains in which to develop systems. This led, perhaps indirectly, to the *RoboCup* initiative [Kitano, 1998; RoboCup, 2001]. The RoboCup challenge is simple: to demonstrate, within 50 years, a team of soccer-playing robots that can beat a World-Cup-strength team of human soccer players. The rationale for RoboCup is that successfully playing soccer demands a range of different skills, such as real-time dynamic coordination using limited communication

bandwidth. (From a robotics point of view, RoboCup also presents profound challenges – today’s autonomous robots come nowhere near the dexterity or flexibility of human soccer players.) Interest in RoboCup had exploded by the turn of the century, with hundreds of teams from across the world attending the regular RoboCup tournaments. In 2000, RoboCup launched a new initiative, entitled *RoboCup Rescue*. In RoboCup Rescue, the goal is to build robots that can cooperate to carry out search and rescue missions in a scenario based on the earthquake that hit the city of Kobe, Japan, in the mid 1990s. Miniature robots designed by a team working on the RoboCup Rescue scenario were used to search in the ruins of the World Trade Center in New York, following the devastating terrorist attacks of 11 September 2001.

Since 2001, ideas from multiagent systems have taken off in mainstream computer science. In particular, auctions have become a major research topic in theoretical computer science, and the area of *mechanism design* is, at the time of writing (summer 2008), one of the most active research topics in theoretical computer science. There are several reasons for this explosion of interest. Certainly the enormous success of online auction houses such as eBay is one reason. But more generally, the interest arises from the fact that many interesting and important combinatorial problems can be understood as auctions. There has also been a rapid growth of interest in the connections between game theory and computer science in general. This growth of interest was recognized by the Game Theory Society, who in 2008 initiated a new award for the best research at the intersection of game theory and computer science. The inaugural award was won by Daskalakis, Goldberg, and Papadimitriou for their work on classifying the complexity of Nash equilibria.

¹ To many researchers who do ‘good old-fashioned AI’, the title of this paper – *Intelligence without representation* – is provocative, if not actually heretical.

² The notion of objects communicating by message passing was a key idea in early object-oriented programming systems, but has been somewhat obscured in languages such as C++ and Java. Message passing in Smalltalk was essentially method invocation.

Appendix B Afterword

I began this book by pointing to some trends that have so far marked the short history of computer science: ubiquity, interconnection, intelligence, delegation, and human-orientation. I claimed that these trends naturally led to the emergence of the multiagent systems paradigm. I hope that after reading this book, you will be able to agree with this claim.

After opening this book by talking about the history of computing, you may expect me to close it by talking about its future. But prediction, as the saying goes, is hard – especially predicting the future. Rather than making specific predictions about the future of computing, I will therefore restrict my observations to some hopefully rather uncontentious (and safe) points. The most important of these is simply that these trends will continue. Computer systems will continue to be ever more ubiquitous and interconnected; we will continue to delegate ever more tasks to computers, and these tasks will be increasingly complex, requiring ever more intelligence to successfully carry them out; and, finally, the way in which we interact with computers will increasingly resemble the way in which we interact with each other. Those of us old enough to have worked with computers before 1993 will recall the sense of awe as we realized what might be possible with the World-Wide Web. But this pales into insignificance next to the possibilities of this, as yet distant future, Internet.

The *plumbing* needed to build this future – the processors and the network connections to link them – is, to all intents and purposes, already in place. The difficult part – the real challenge – is the software to exploit its potential. I do not know exactly what software technologies will be deployed to make this future happen. But it seems to me – and to many other researchers – that multiagent systems are the best candidate we currently have. It does not matter whether we call them agents or not; in 20 years, the term may not be used. The key thing is that the problems being addressed by the agent community – and discussed in this book – are exactly the problems that I believe will need to be solved to realize the potential.

Glossary of Key Terms

3t architecture

A hybrid, layered agent architecture

ABox

The ‘assertion box’ in a description logic system: think of it as a set of facts, as distinct from axioms.

Abstract argumentation

A framework for argumentation, developed by Dung, which does not consider the internal structure of arguments: each argument is an atomic entity, and an ‘attack’ relation holds between these arguments.

Accessibility relation

A key concept in ‘Kripke semantics’ for modal logic: a relation over the set of possible worlds that an agent considers possible, characterizing what the agent believes or knows.

Achievement tasks

A task in which an agent has to achieve or bring about some state of affairs.

Acquaintance models

A data structure capturing what an agent knows about its peers.

Activity-producing layers

A key concept in layered agent architectures: an agent is composed as a hierarchy of layers, each of which is independently operating (i.e., each is ‘activity producing’).

Ad hoc ontologies

An ontology constructed without a formal basis or semantics: often constructed using, for example, XML.

Add list

In the STRIPS notation for representing actions, the add list defines the things that become true after the action is performed.

Additivity axiom

One of the three axioms defining the Shapley value in cooperative game theory: says that the payoff an agent gets from the sum of two games is the sum of the payoffs it gets in the component games.

Adjustable autonomy

The idea that we have some control over how autonomous an agent is: for example we

The idea that we have some control over how autonomous an agent is. For example, we might specify that an agent should verify certain decisions with us before proceeding with them.

Admissible set

A set of arguments in an abstract argument system is admissible if it is internally consistent (i.e., no argument in the set attacks any other argument) and defends itself against all attackers.

Agent architectures

A software architecture for autonomous decision making: specifies the data structures, control flow, and, usually, the methodology to be used for constructing an agent for a particular task in a particular environment.

Agent class hierarchy

In agent-oriented software engineering, specifies a set of agent types that inherit properties, in much the same way as an object class hierarchy as object-oriented programming.

Agent instance model

In agent-oriented software engineering, specifies the instances of agent classes that exist in a run-time system.

Agent synthesis

The process of constructing an agent for a particular task environment.

Agent wrappers

The idea that we can ‘wrap’ a legacy software component in an ‘agent layer’ so that legacy code can operate as an agent.

Agent-oriented decomposition

In agent-oriented software engineering, a technique for identifying the agents to build for a system.

Agent-oriented programming

The idea of developing distributed systems as collections of interacting (semi-)autonomous, rational software agents – Yoav Shoham’s AGENT0 language was the first instance of this concept, and was the first paper to make this idea explicit.

AGENT0 language

An agent-oriented programming language, developed by Yoav Shoham in the late 1980s/early 1990s: the language took seriously the idea of programming agents as rational entities, with beliefs about how the world is, and commitments about how the agents wanted the world to be – an AGENT0 agent acts rationally to achieve its commitments on the basis of its beliefs about how the environment is.

Algorithmic game theory

A part of theoretical computer science that studies computational aspects of game theory, such as the complexity of computing solution concepts, and the applications of mechanism design in computer science.

Alternating offers

A bargaining/negotiation protocol developed by Ariel Rubinstein, in which agents take it in turns to make proposals and counter proposals.

Application ontology

An ontology whose domain is a specific application area, rather than attempting to capture general knowledge about the environment.

Approximate winner determination

In combinatorial auctions, the winner determination problem is a computationally intractable problem, involving finding an optimal allocation of goods to bidders: in approximate winner determination, we aim to find an allocation that may not be optimal, but is within some known bound of the optimal.

Arrow's theorem

A key negative result in social choice theory: crudely, the theorem says that no social choice (voting) procedure can satisfy a certain collection of desirable properties.

Ascending auction

An ascending auction is one in which bids start low and get progressively higher.

ATL

Alternating-time Temporal Logic (ATL) is a logic intended to support reasoning about the strategies powers of agents and coalitions of agents in game-like distributed systems.

Atomic bids

In combinatorial auctions, a key problem is that of concisely expressing the preferences that agents have over bundles of goods: bidding languages are used to express these preferences, and are composed of atomic bids, typically using OR or XOR operators.

Attack relation

In an abstract argument system, the attack relation represents incompatibilities between arguments: if argument α_1 attacks argument α_2 , then if α_1 is accepted then α_2 should not be.

Auction bots

Simple software agents that bid on behalf of their owner in an online auction.

Autonomic computing

The idea of building software systems that have the capability to monitor themselves and autonomously repair and adapt themselves as required.

Autonomy/autonomous action

For the purposes of this book, the ability of a piece of software to determine for itself how to achieve its delegated goal/task.

Average marginal contribution

The Shapley value defines the payoff assigned to an agent to be its average marginal contribution considering all possible coalitions, formed in all possible orders.

Award and inform processing

A key stage in the contract net task sharing protocol: handling the awarding of tasks to agents, and informing agents of their success or otherwise in bidding for tasks.

Axelrod's tournament

A tournament organized by political scientist Robert Axelrod in the early 1980s, to investigate the performance of strategies in the iterated prisoner's dilemma: the winner of Axelrod's tournament was TIT-FOR-TAT (which does *not* mean TIT-FOR-TAT is optimal strategy in the prisoner's dilemma).

Axioms

The basic logical 'givens' in a logical theory.

Backward induction

The process of determining how best to act in a game by starting at the last state, assuming we are in that state, determining the best action for it, and then filtering this information to the previous state, and so on.

Banzhaf index

A measure of power in voting games, closely related to the Shapley value.

Behavioural agent

An agent that decides what to do without explicit abstract/symbolic reasoning (sometimes called reactive agent).

Belief database

A set of statements, typically expressed in logic, capturing the information that an agent has about its environment (beliefs may be incorrect, which distinguishes them from *knowledge*).

Believable agents

An agent in a simulated environment that acts in such a way as to cause the 'suspension of disbelief' in the scenario.

Benevolence assumption

The assumption that agents will always be helpful, will always tell the truth: in general, where agents have conflicting preferences or goals, we cannot make the benevolence assumption.

Best response

An action for me is a best response to an action of yours if there is no other action for me that would result in a more preferred outcome for me.

Bid processing

One of the key stages of the contract net protocol: upon receipt of bids for a task by multiple agents, the process of choosing between them.

Bidding languages

In combinatorial auctions, a language that is used by bidders to express their valuation functions – OR and XOR languages are the best known.

Binary voting tree

A binary tree in which leaf nodes are labelled with candidates in the election.

Blank node

In RDF, a blank node is a technical trick used to represent information that cannot be directly represented.

Blind commitment

A type of commitment that an agent has towards an intention.

Blocking communication

Communication between processes, as typified by method invocation, in which the initiator blocks (i.e., waits) until the recipient responds.

Bold agent

An agent that does not frequently reconsider intentions.

Borda count

A voting protocol that takes into account the entire preference order of every agent when computing winner.

Boulware

A common strategy in bargaining/negotiation, in which concessions are made only at the last possible moment.

Bounded optimal agent

Bounded-optimal agent

An agent that behaves as well as any agent could behave given the computational resources available to it.

C-contracts

In Sandholm's bargaining over resource allocation framework, a C-contract is a 'cluster' contract.

Calculative rationality

An agent enjoys the property of calculative rationality if it chooses the action to perform that is optimal under the assumption that the world has remained unchanged since it observed it.

Cautious agent

An agent that frequently reconsiders its intentions.

CDPS

Cooperative Distributed Problem Solving.

Characteristic function

The characteristic function v of a cooperative game assigns to every coalition C a real value $v(C)$ defining the value that this coalition could obtain should they choose to cooperate with one another.

Coalition

A group of agents.

Coalition structure generation

The process of dividing the set of all agents into exhaustive disjoint coalitions (i.e., a partition), typically with the goal of maximizing the sum of values of coalitions in the partition.

Coalition structure graph

A representation of possible coalition structures used in the computation of a coalition structure.

Coalitional resource game

A scenario in which each agent has goals to achieve, and is endowed with certain amounts of resources; coalitions form to pool resources to achieve mutually satisfying sets of goals.

Commissives

A type of speech act, used by an agent to communicate commitments (e.g., promising).

Commitment

a.k.a. intentions: something the agent has resolved to achieve.

Commitment rules

In the AGENT0 programming language, commitment rules define how an agent should update its commitments based on messages received and beliefs about the world.

Common value

A type of auction in which every participant values a good in the same way.

Comparison shopping

Comparing the price (and possibly other attributes) of a particular good as offered by a number of vendors.

Complete representations

A complete representation for coalitional games is a representation that can capture any arbitrary coalitional game.

Complexity of manipulation

How hard it is to determine how to optimally vote in an election.

Conceder

A strategy used in bargaining or negotiation, in which concessions are made very quickly.

Concept satisfiability

In description logic, the problem of checking whether it is possible for a class/concept to have any instances.

Concrete/abstract syntax

In ontology languages, the distinction between the actual language used to represent an ontology for processing by computer, and the language used to represent it typically for reading by people.

Concurrent MetateM

A programming language for multi-agent systems in which agents are programmed by giving them temporal logic formulae, which are directly executed in order to generate behaviour.

Condorcet winner

A Condorcet winner is a winner that would beat every other candidate in a pairwise election.

Condorcet's paradox

The fact that there are situations in which no matter which candidate wins an election, another candidate would have been preferred by a majority of candidates.

Conflict deal

In bargaining or negotiation, a conflict deal is the outcome that will be implemented if agreement is not reached.

Conflict free argument set

In abstract argument systems, a set of arguments is conflict free if no argument within the set attacks any other argument within the set.

Continuous double auction

An auction in which participants buy and sell simultaneously – used in the Trading Agent Competition.

Contract net

A classic protocol for task sharing, modelled on the process of putting contracts out for tender in human businesses.

Control subsystem

In layered agent architectures in general (and TouringMachines in particular), the control subsystem mediates between the different layers, deciding which layer should have control..

Controlled vocabulary

A simple kind of informal ontology, in which certain terms are reserved for certain purposes.

Cooperation logics

A logic intended to support reasoning about cooperation: ATL is perhaps the best-known example.

Cooperation modalities

The basic cooperation expression of ATL.

Cooperative games

A game in which binding agreements are assumed to be possible.

Cooperative problem solving

Solving a problem by distributed semi-autonomous agents, typically assumed to share a common goal.

Coordinated attack

A famous problem in the distributed systems literature, where two generals will only attack if it is common knowledge between them that they will both attack: if communication is unreliable, common knowledge is impossible to achieve, no matter how many acknowledgements are sent.

Coordination

The problem of managing the interactions between the actions of multiple agents.

Coordination relationships

In von Martial's typology of interactions between actions, coordination relationships capture ideas such as one action enabling or disabling another.

Core

A key solution concept in cooperative games: the core is the set of outcomes to which no coalition has any objection, in the sense that they could not do better themselves – if the core is non-empty, then the coalition is said to be stable, since no sub-coalition has any incentive to deviate and form a coalition on their own.

Correct plan

A plan is correct with respect to a goal if, after it is executed, the goal will be achieved.

Correlated value

A type of auction in which agent's valuations are related to one another, but not necessarily common value.

Correspondence theory

In modal logic, correspondence theory studies the relationship between the properties of models and the axioms that these properties make true (correspond to).

Credulous acceptance

In abstract argument systems, an argument is credulously accepted if it is in at least one preferred extension.

Declarations

In speech acts, a declarative speech act is an action such as declaring war or declaring a couple to be man and wife: typically carried out in a formalized setting.

Decoy tasks

In task-oriented bargaining/negotiation, an agent might declare a decoy task in order to mislead the bargaining partner about its true tasks.

Deduction rules

In logic, a deduction rule is a rule of inference.

Deductive argument

A type of argument that has some logical structure, and in which notions such as attack between arguments are related to logical deduction.

Defended argument

An argument such that, for every attack on the argument, there is a counter attack.

Delegation

The idea that we delegate goals/tasks to agents.

Delete list

In the STRIPS notation for representing actions, the delete list defines the things that become false after the action is performed.

Deliberate agents

An agent that decides what to do via logical/symbolic reasoning.

Deliberation and planning

The components of practical reasoning: deliberation is the process of deciding which goals to achieve, planning is the process of figuring out how to achieve these goals.

Description logic

A knowledge representation formalism underpinning ontology languages such as OWL.

Design stance

The idea that we understand the behaviour of a complex entity by considering its design.

Desire filtering

The process of choosing between competing options, to choose some consistent subset of these, which then become intentions: things an agent is committed to achieving.

Dialogue

An exchange of messages between communication participants.

Dictatorships

A voting protocol is a dictatorship if the outcome of the protocol is determined by one particular agent.

Directed contract

A variant of the contract net protocol.

Discount factor

A value that indicates how the utility of a bargaining outcome decays over time.

Distributed knowledge

Something is distributed knowledge within a group of agents if it can be derived by pooling

the knowledge of agents within the group.

Distributed sensing

An application domain for multiagent systems, in which a group of agents cooperate to monitor activity within some environment.

Deliberation

The process of choosing intentions.

Domain ontology

An ontology relating specifically to an application domain.

Dominant strategies

A strategy is dominant if there is no other action that yields a more preferred outcome.

DTD

Document Type Definition: in SGML and XML, the definition of a set of tags and document structure.

Dummy player axiom

One of the three axioms characterizing the Shapley value, which says that an agent that never adds any value to a coalition receives only the utility that it could obtain on its own.

Dung-style argument system

A type of argument system in which arguments are assumed to have no internal structure.

Dutch auction

A type of auction in which bidding starts with a high value, and is then reduced by an auctioneer until some participant is willing to bid.

DVMT

The Distributed Vehicle Monitoring Testbed – an early multi-agent distributed sensing system.

Election agendas

The order in which candidates will come up against each other in an election.

Emergent behaviour

The idea that a particular property is exhibited by some collection of system components, but not by any individual components: the behaviour ‘emerges’ from the interaction of the components.

Encounter

A setting in task-oriented negotiation.

English auctions

A type of auction in which bidding starts at some low reserve value, and then bidders submit increasing bids until no bidder is willing to go higher.

Episodic environments

A type of task that contains of a number of discrete ‘episodes’ which do not interact with each other – what an agent does in one episode has no influence on other episodes, so the consequences of action are limited to the episode where the action was performed.

Epistemic alternatives

A set of possible states of affairs characterizing what an agent knows: the agent knows ϕ if ϕ is true in all epistemic alternatives.

Executable specification

The idea of directly executing a formal specification of an agent’s behaviour, an idea embodied in languages such as Concurrent MetateM.

Expected utility

The value/payoff that an agent can expect to earn on average.

Expressives

A type of speech act, expressing some property of their mental state.

FA/C

Functionally Accurate/Cooperative.

Fair division

The problem of dividing some resource amongst agents in a fair way.

Favour

A type of coordination relationship: an action favours another if it enables it, or improves it in some way.

Felicity conditions

In speech act theory (and in particular the work of Searle), conditions defining when a speech act could be said to have truly taken place.

FIPA

Foundation for Intelligent Physical Agents – a standardization organization for agent systems, particularly agent communication languages.

First-price auction

A type of auction in which the amount the auction winner pays for a good is the amount of the highest bid.

Focal state

In social laws, a focal state is a state that an agent wants to be able to visit without being prevented from doing so.

Free disposal

The idea that having more of a particular resource is never worse than having less.

Functional system

A system such as a compiler, which can be understood as a function $f: D \rightarrow R$ from some domain of values D to some range R – in contrast to a reactive system.

Future-directed intentions

Intentions directed towards future events, in contrast to, e.g., saying that you performed an action intentionally, where intention characterizes an immediate action.

Game dimension

The minimum number of individual weighted voting games required to construct a yes/no voting game.

Game of chicken

A particular type of 2×2 game with two Nash equilibria, (C, D) and (D, C) .

General game playing

The idea of being able to play arbitrary games after downloading and interpreting their rules

Gibbard–Satterthwaite theorem

A fundamental result in social choice theory, which very crudely says that voting procedures are prone to strategic manipulation.

GPGP

Generalized Partial Global Planning – an approach to coordination in multiagent systems.

Grand coalition

The set of all agents in a game or system..

Grounded extension

A solution concept for Dung-style abstract argument systems.

Grounding

The idea that the semantics of an agent logic should be given a clear computational interpretation.

Heuristic winner determination

In combinatorial auctions, finding an optimal allocation (winner determination) is computationally complex – heuristic approaches aim to find a (not necessarily optimal) allocation quickly, using rules of thumb (heuristics).

Hidden tasks

In task-oriented bargaining/negotiation domains, a hidden task is one that an agent conceals in order to obtain a more favourable outcome – an example of strategic manipulation.

Horizontal layering

A type of layered hybrid agent architecture, where each subsystem has access to sensor input and directly produces suggestions for actions to perform.

Human-orientation

The idea that the way in which we interact with a computer system ever more closely resembles the way in which we interact with humans.

IIA

Independence of Irrelevant Alternatives – one of the desirable properties for voting procedures considered in Arrow's theorem.

Illocution

A term usually used to refer to a speech act.

Implementation theory

The part of game theory concerned with designing games to satisfy certain properties.

Incentive compatible

A voting protocol (or more generally, social choice mechanism) is incentive compatible if no agent can benefit by misrepresenting their preferences, i.e., if truth-telling is a rational choice.

Instance classification

In ontology systems, the idea of taking some entity, whose properties are partially known, and classifying this entity, e.g., identifying the most specific class to which this entity belongs.

Integer linear programming

A type of optimization problem, in which we want to find integer values for some unknown variables, where constraints on these variables are expressed as linear inequalities over them.

Intentional stance

The idea of explaining the behaviour of agents by attributing mental states like belief and desire together with rational acumen.

Intentional system

A system that can be explained by the intentional stance.

Intentions

States of affairs that an agent is committed to bring about.

Interpreted symbolic structures

A model of belief in which an agent has a set of logical formulae, closed under some set of deduction rules.

Interpreted system

A formal model of knowledge, in which two system states are considered to be epistemic alternatives for some agent if the agent has the same internal state in both.

Interrap architecture

A vertically layered hybrid agent architecture.

Iterated prisoner's dilemma

A game in which players play the prisoner's dilemma a number of times.

JADE container

A programming construct used in the JADE Java-based implementation of the FIPA agent communication language.

KIF

The Knowledge Interchange Format – essentially, a standardized version of first-order logic, used for defining both ontologies and message content.

Knowledge base

An ontology together with some instances.

KQML

The Knowledge Query and Manipulation Language – an agent communication language.

Layered architecture

A type of agent architecture in which different behavioural components are arranged into layers, typically with lower-level layers responsible for more primitive types of behaviour, and taking priority over layers higher up.

Least common subsumer

The most specific ancestor class in an ontology.

Legacy software

Software that is essential to an organization, but which was written using antiquated development techniques; ‘agent wrappers’ can be used to embed legacy software within an ‘agent layer’, making it possible to allow the software to interact with other agents.

Locutionary act

A speech act.

Logical omniscience

In modal epistemic logic, the undesirable property that an agent knows the logical consequences of its knowledge.

Logical specification

A logical description of the desirable properties of a system – temporal logic is a commonly used logical specification language.

M-contracts

In Sandholm’s bargaining over resource allocation framework, an M-contract is a ‘multiagent’ contract, involving at least three agents.

Maintenance task

A maintenance task is one in which an agent must maintain some state of affairs.

Marginal contribution net

A representation for coalitional games, based on rules with the form ‘condition $\rightarrow$ value’.

Marginal contribution

In coalitional game theory, the amount that an agent adds to a coalition by joining it.

Marginal cost of task

The cost involved in taking on a new task.

Markup

In SGML-related web standards (HTML, XML, ...), markup refers to the tags used to indicate how web content should be formatted.

Maximal argument set

An argument set to which no further arguments can be added without the set losing some desirable property.

Means-ends reasoning

In practical reasoning, the process of planning, i.e., determining a plan or recipe to take the agent from the current state of the world to a state where a desired goal is achieved.

Mechanism design

The process of designing a game so that, if the participants in the game act rationally, some desirable overall objective will be achieved.

Mediating act

In the plan-based theory of speech acts, an act that links the hearing of a speech act to its desired effect.

Mediator

In program equilibria, the central component interpreting the program strategies submitted by players.

Mental condition

In commitment rules in the AGENT0 programming language, the mental condition part of a rule is a condition on the current beliefs of an agent.

Mental state

The cognitive makeup of an agent: its beliefs, desires, intentions, hopes, fears,...

Message condition

In commitment rules in the AGENT0 programming language, the message condition part of a rule is a condition on the messages received by an agent.

Meta search

A meta search engine is a web search engine that uses the results of other web search engines.

Meta-language

A language that can refer to other languages – logical meta-languages have terms that denote terms of another language, often in a hierarchical fashion.

Meta-level control

Having control over the control structures of an agent.

Meta-level plans

Plans that contain actions to manipulate the plans of the agent.

Micro/macro levels

The distinction between agent-level (micro) and societal-level (macro) reasoning.

The distinction between agent level (micro) and society level (macro) concerns.

Middle agents

Agents that have knowledge about the knowledge/expertise/abilities of other agents.

Middleware

Software that sits between an agent and some resource, providing access to it.

Mixed strategy

In game theory, a mixed strategy is a strategy that randomizes over possible choices.

Mobile agent

An agent that can transmit itself over an electronic network, recommencing execution at some remote site.

Modal logic

A type of non-classical logic containing non-truth functional operators that 'change the mode' of interpretation – the semantics to modal logics are classically Kripke semantics.

Modal operator

A non-truth functional operator in modal logic.

Modelling layer

In the TouringMachines agent architecture, the modelling layer is responsible for modelling other agents in a system.

Modular representations

A representation for coalitional games that achieves succinctness by decomposing a representation of a game down into a number of smaller games – marginal contribution nets are one example.

Monotonic concession protocol

A bargaining/negotiation protocol.

Multiagent systems

A system containing multiple agents.

Mutually defensive argument set

In Dung-style abstract argument systems, a set of arguments is mutually defensive if for every attack on some argument in the set, there is a counter attack by some member of the set.

Namespace

In ontology languages, the scope of a defined vocabulary.

Nash equilibrium

A fundamental solution concept in game theory: a collection of strategies forms a Nash equilibrium if every strategy in the set is a best response to the other strategies – no agent has any incentive to deviate from a Nash equilibrium.

Nash's theorem

Every game with finite strategies has a Nash equilibrium in mixed strategies.

Necessary/contingent truth

A necessary truth is one that could not possibly be false (e.g., a mathematical law such as $1 + 1 = 2$), while a contingent truth is one that could possibly be otherwise (e.g., it is raining in Liverpool).

Necessitation

A rule of inference in modal logic: for example, in modal epistemic logic, if φ is valid, then φ is known.

Negative introspection

The idea that an agent is aware of what it doesn't know; in modal epistemic logic, negative introspection is defined by axiom 5.

Negotiation decision function

In bargaining/negotiation, a function defining the offers that an agent makes as a function of time.

Negotiation set

The set of possible proposals that an agent is allowed to make.

Network flow game

A coalitional game interpreted on network flows, in which a coalition gets a payoff of 1 if they can ensure a flow of k from a source node s to a sink node t .

Normal modal logic

A type of modal logic containing the 'K' axiom.

O-contracts

In Sandholm's bargaining over resource allocation framework, an O-contract is a 'one' contract, involving a single resource.

Objective function

In linear programs, the objective function defines what is to be maximized (or minimized).

One shot auction

An auction in which there is a single round of bids.

One-pass architecture

A type of layered agent architecture, in which sensor data enters in one layer, then is processed by progressive layers until decisions are made in a final layer.

Ontology

A formal specification of some domain of interest.

Ontology reuse

The idea of being able to reuse ontologies for multiple applications.

Opaque context

Concepts such as belief, desire, knowledge set up opaque contexts in the sense that the truth value of a sentence such as 'Lily believes it is raining' cannot be computed from the truth value of its components.

Open cry

A type of auction in which every agent gets to see the bids of every other agent.

Optimal agent

An agent is optimal if it maximizes expected utility.

OR bids

A type of bidding language for combinatorial auctions, in which a valuation function is defined as the 'disjunction' of a number of atomic bids.

Organizational design

The problem of designing a society of agents.

OWL DL

A variant of the OWL ontology language, corresponding to a particular fragment of description logic.

OWL Full

An expressive variant of the OWL ontology language.

OWL Lite

A comparatively weak fragment of the OWL ontology language.

Pairwise election

An election in which there are only two candidates, and in which the winner is the one receiving the majority of the vote.

Parallel problem solving

The idea of exploiting parallelism in problem solving.

Pareto efficient/optimal

An outcome is said to be Pareto efficient if there is no other outcome that would improve the lot of one agent without making another agent worse off – if an outcome is *not* Pareto efficient, then it ‘wastes utility’, since we can make somebody better off without making anybody else worse off.

Partial global planning

A coordination regime for cooperative problem solving.

Payoff matrix

A representation for (two player) games, with a row player and a column player: the rows represent strategies for the row player, columns represent strategies for the column player, and cells in the matrix represent outcomes.

Payoff vector

In coalitional game theory, a vector of payoffs, one for each player, defining their share of the benefits of cooperation.

Percepts

The ‘sensor data packages’ received by an agent.

Performatives

In agent communication, performatives are essentially, message types.

Perlocution

The intended effect of a speech act.

Permissions

In agent-oriented software engineering, the rights that agents have in order to achieve their goals.

Phantom tasks

In task-oriented negotiation, an agent can declare phantom tasks in an attempt to misrepresent its state.

Physical stance

The idea of explaining or predicting the behaviour of a complex system by reference to its

design or structure – contrast with the intentional stance.

Physically embodied agent

An agent such as a robot, whose decision-making and action directly affect the physical world.

Plan library

The idea that instead of planning from first principles, an agent has a library of pre-compiled plans/recipes.

Planning layer

In the TouringMachines agent architecture, the planning layer has overall responsibility for selecting and executing plans in the furtherance of the agent's goal.

Planning problem

A planning problem is defined by a representation of the current state of the environment, a goal/task to be achieved, and a set of possible actions – the goal is to find a plan/recipe such that, when executed from the current state, will lead to a state in which the goal/task is satisfied.

Plans/recipes

A plan is essentially just a program: think of it as a list of actions to perform.

Plurality voting

A voting system in which a number of candidates stand, and the winner is the one with the largest proportion of the votes cast.

Positive introspection

The idea that if an agent is aware of what it knows; in modal epistemic logic, negative introspection is defined by axiom 4.

Possible worlds

A framework for giving a semantics to modal logic: something is said to be necessary if it is true in all possible worlds (a.k.a. Kripke semantics).

Practical reasoning

Reasoning directed towards action: the process of figuring out, moment by moment, what action to perform next – often assumed to consist of deliberation and means ends reasoning.

Precondition list

In the STRIPS notation for representing actions, the precondition list defines the things that must be true in order for the action to be executed.

Predicate task specification

A way of specifying tasks for agents, in which we define a task as a predicate over runs: an agent is ‘successful’ on a run if that run satisfies the predicate.

Preference aggregation

The fundamental problem of determining a ‘social preference order’ based on the possibly conflicting preferences of an electorate.

Preference-based argumentation

A variant of Dung-style abstract argument systems, in which as well as an attack graph on arguments, we also have a preference ordering over them.

Preferred extension

In Dung-style abstract argument systems, a preferred extension is a conflict free set of mutually defensive arguments, to which no argument can be added without the argument set failing to satisfy these properties.

Prisoner’s dilemma

A fundamental 2×2 game, in which mutual defection is the rational course of action, despite the fact that mutual cooperation would lead to an outcome that would be preferred by both players.

Private action

An action that is ‘internal’ to an agent – think of the agent executing a sub-routine, for example.

Private value

A type of auction in which agents have their own valuation for goods, which they do not necessarily share with others.

Pro-attitudes

Mental states such as desires or intentions, which tend to lead to action.

Proactivity

Goal-directed behaviour; taking the initiative; not being driven solely by events.

Program equilibria

The idea that certain outcomes can be reached in games like the prisoner’s dilemma if they are allowed to include program strategies that are conditioned on the programs submitted by others (e.g., ‘if his program is the same as mine then cooperate otherwise defect’).

Program rules

In the Concurrent MetateM language, program rules are the past $\rightarrow$ future temporal logic rules used to define the agent’s desired behaviour.

Proxy bidding

The idea of having a proxy (a program or a person) submit bids on our behalf in an online auction.

PRS

The procedural reasoning system – a well-known belief–desire–intention agent architecture, in which agents dynamically select plans from a plan library in order to achieve their intentions, based on the current context in which they find themselves.

Pure strategy Nash equilibrium

A Nash equilibrium in which the probabilities associated with playing strategies are either 0 or 1.

Purely reactive agent

An agent that does no reasoning whatsoever: it simply maps a current situation to corresponding action.

Qualitative coalitional game

A type of coalitional game in which agents have goals to achieve, and different coalitions can achieve different sets of goals: coalitions form to achieve mutually satisfying sets of goals.

Rational balance

The characterization of a rational mental state, for example, in which an agent intending φ believes that φ is possible.

Rational effect

What an agent hopes to achieve by performing a speech act.

Rationally justifiable position

The solution to an argumentation problem.

RDF

The Resource Definition Framework – a simple knowledge representation language intended for representing knowledge about web resources – based on semantic networks.

RDF triples

The basic data structure in RDF, containing a subject, predicate, and object.

Reactive layer

In the TouringMachines agent architecture, the modelling layer is responsible for low-level behaviours such as obstacle avoidance.

Reactive system

A system that maintains an ongoing interaction with its environment.

Reactivity

The ability of an agent to be aware of and respond to changing environmental circumstances.

Rebuttal

An argument rebuts another argument if it directly attacks the conclusion of the argument.

Remote procedure call

A classic blocking form of communication/interaction in distributed systems.

Representatives

A type of speech act.

Responsibilities

Associated with a role in agent-oriented software engineering, responsibilities define the tasks associated with the role.

Result sharing

A form of cooperative problem solving, in which a partial solutions to problems are shared by agents.

Roles

A key idea in agent-oriented software engineering: a role is defined by responsibilities/obligations and permissions/rights.

Runs

A computation of an agent/environment system: an interleaved sequence of environment states and actions.

S-contracts

In Sandholm's bargaining over resource allocation framework, an S-contract is a 'swap' contract, involving the swap of a single resource.

Safe TCL

A mobile-agent programming language, based on the TCL scripting language.

Sceptical acceptance

The idea that an argument is accepted only if it is a member of every preferred extension.

Sealed bid

A type of auction: in a sealed bid auction, bidders submit bids in private to the auctioneer (in contrast to open cry auctions).

Second price auction

An auction in which the winner of the resource pays the amount of the second highest bid (a.k.a. Vickrey auction) – bidding truthfully is a dominant strategy in second price auctions.

Second-order Copeland

A voting protocol that is computationally hard to manipulate.

Semantic markup

The idea of using tags to markup documents on the world-wide web that convey information about the meaning/content of the document.

SGML

Standard Generalized Markup Language – an international standard for defining structured documents; HTML is defined using SGML, and XML can be thought of as a cut-down version of HTML.

Shapley value

A solution for coalitional games in which the payoff an agent receives is its average marginal contribution – the Shapley value is the unique solution to three fairness axioms.

Shapley–Shubik index

A variant of the Shapley value used in voting systems to measure the power of players.

Side payments

The idea that agents can be compensated for accepting less desirable outcomes by giving them separate payments.

Simple games

A coalitional game in which every coalition gets either the value 1 (in which case we say they are winning) or 0 (in which case we say they are losing).

Simple majority voting

A voting system in which the winner of a ballot is the one with a simple majority.

Situated agent

An agent that inhabits (is situated in) some environment.

Situated automata

A framework for building agents developed by Rosenschein and Kaelbling, in which a declarative specification of an agent is ‘compiled’ into an executable automata-like form.

Slater rule

A voting system that tries to minimize inconsistencies in the social ranking produced.

Sniping

In an online auction such as eBay, sniping is the behaviour whereby bidders try to submit bids at the last possible moment.

Social ability

The ability of an agent to engage in analogues of the social processes that we all engage in every day: not just communication, but cooperation, coordination, and negotiation.

Social choice

Social choice theory is that part of economics concerned with decision making in settings where participants have potentially conflicting preferences.

Social laws

A social law is a set of constraints on the behaviour of agents, designed so that if these constraints are adhered to, then some overall desirable objective results.

Software agent

An agent that inhabits a software environment, as opposed to a physically embodied agent.

Speech act theory

A pragmatic theory of speech, which views utterances as actions, and considers the way in which agents use language to achieve their goals.

Stability

A particular strategic scenario is said to be stable if no participant has any incentive to deviate from it.

Stag hunt

A particular type of 2×2 game.

Strategic manipulation

A social choice procedure, or voting procedure, is said to be prone to strategic manipulation if it is in principle possible for a voter to obtain a better outcome by misrepresenting their preferences.

Strategic powers

An agent's strategic powers relate to what states of affairs it can bring about by its actions.

Strategy update function

A function that tells an agent how to modify its strategy on the basis of what it has witnessed so far.

Stratified argument set

A hierarchical version of Dung's abstract argument systems, in which arguments in higher layers are stronger than arguments in lower layers.

Strictly competitive

A scenario is strictly competitive if the better outcomes for one participant are worse for another, and vice versa.

STRIPS notation

A symbolic representation for actions in planning systems, in which an action is characterized in terms of a pre-condition, an add list, and a delete list..

Subclass/superclass

A basic relationship between classes in ontology frameworks: if A is a subclass of B , then A typically inherits all the properties of B .

Subsumption architecture/hierarchy

A classic behavioural/reactive agent architecture developed by Rodney Brooks – the basic idea is to arrange task-accomplishing behaviours into a hierarchy, with layers lower down in the hierarchy taking priority over those further up; more generally, behaviours can interact with one-another in richer ways.

Succinct representations

In coalitional games, naive representations tend to be exponentially large in the number of agents – we therefore usually seek more compact, or succinct representations, requiring space polynomial in the number of agents.

Symmetry axiom

One of the three axioms defining the Shapley value in cooperative game theory: it says that agents making the same contribution should get the same payoff.

Synchronization

The problem of designing the interactions between processes, typically to ensure that they do not destructively interfere with one another.

TAC

The Trading Agent Competition – a competition intended to test the ability of computer programs to automatically trade goods and services.

Tactical voting

A form of strategic misrepresentation in voting: if you feel your candidate has no chance of

A form of strategic misrepresentation in voting. If you feel your candidate has no chance of winning, you vote for somebody else who has a higher chance of winning, even though this is not your most preferred outcome.

Task announcement

In the contract net protocol, a task announcement is sent out by a prospective manager to potential bidders, defining the task that is to be carried out.

Task sharing

An approach to cooperative problem solving in which an overall task is decomposed into smaller tasks, which the 'task manager' then advertises to agents, who can then bid to carry out the tasks.

Task specification

A high-level description of a task to be carried out.

Task-accomplishing behaviour

In the subsumption architecture, an agent is constructed as a hierarchy of individual components, each of which tries to accomplish some specific independent task.

Task-oriented domain

A domain for bargaining/negotiation first formalized by Rosenschein and Zlotkin, where agents have tasks to carry out, and can mutually benefit from re-distributing these tasks amongst themselves: the goal of bargaining is to find such a re-allocation.

TBox

In ontology systems, the TBox is the terminological box, typically defining the classes of the ontology and their relationships.

Telescript agents

Mobile software agents implemented in the Telescript language.

Temporal logic

An extension of classical logic intended for reasoning about how the truth of propositions change over time.

Theoretical reasoning

Reasoning directed towards beliefs (in contrast to practical reasoning).

TIT-FOR-TAT

The winning strategy in Axelrod's prisoner's dilemma tournaments: cooperate on the first round, and then on subsequent rounds do what the other player did on the preceding round.

Top-down refinement

A classic development methodology for software development, well suited to developing functional programs.

Total search problem

A search problem in which we know there is a solution (e.g., computing a mixed strategy Nash equilibrium in a finite game: by Nash's theorem, such a Nash equilibrium will exist) – PPAD is a complexity class associated with such search problems.

TouringMachines architecture

A hybrid, layered agent architecture.

Tournaments

A tournament is a complete directed asymmetric graph, which in voting theory usually has a special interpretation: an edge from ω to ω' means that ω would beat ω' in a simple majority election.

Tragedy of the commons

The problem that a shared free resource will probably be overexploited, despite the fact that it is in everybody's interests not to overexploit it.

Tropistic agents

Another term for purely reactive agents.

Truth functional operators

A logical operator that derives its value solely from the truth values of its arguments.

Two-pass architecture

A type of layered agent architecture, in which sensor data enters in one layer, then is processed by progressive layers until a final layer is reached, at which point decisions are made, which then filter back down the layers.

Ubiquity

The idea that computer processing power is so cheap it becomes possible to put information processing capability everywhere.

Ultimatum game

A game in which one player makes a proposal (classically, about how to divide some pot of money), and the other player can either accept or reject; if they reject, then nobody gets anything – the last stage of alternating offers bargaining can sometimes resemble an ultimatum game.

Undercut

An argument undercuts another argument if it attacks one of the supports for the argument.

Upper ontology

A very general high-level ontology – typically includes classes like ‘thing’.

URI

Uniform Resource Identifier – a standard naming format for identifying web resources.

Useful social law problem

The problem of checking whether there exists a social law that, if implemented, would enable all agents to achieve their goals – identified and proved NP-complete by Shoham and Tennenholtz.

Utility

A numeric measurement of how good an outcome is for an agent: outcomes with higher values are preferred over outcomes with lower values – while it is sometimes useful to think of utility as money, they are not the same thing!

Valuation function

In combinatorial auctions a bidder’s valuation function defines what the bidder is willing to pay for possible bundles of goods.

Value restrictions

In ontology languages, value restrictions place limits on the values that can be taken by attributes.

Value-based argumentation

A variant of abstract argument in which we take account of the values to which an argument appeals.

VCG mechanism

Vickrey–Clarke–Groves mechanism: a generalization of Vickrey auctions used in combinatorial auction settings, which has the property that bidding to one’s true valuation is a dominant strategy (i.e., it is incentive compatible).

Vertical layering

A type of hybrid agent architecture, in which control is divided into a hierarchy of layers, with different layers processing (sensor) information at different levels of abstraction.

Veto player

In coalitional game theory, a player is a veto player if it is a member of every winning coalition.

Vickrey auction

Another name for a second price auction

Another name for a second-price auction.

Virtual knowledge base

In the semantics of KQML-based agent communication, we attribute a virtual knowledge base to an agent in order to be able to talk about it as if it had such a knowledge base, even though we do not require that KQML agents actually explicitly have such a knowledge base.

Voting theory

Part of social choice theory concerned with voting scenarios.

Weighted voting game

A simple coalitional game in which every agent has a numeric weight, and a coalition is deemed to be winning if the sum of their weights exceeds some stated threshold.

Willingness to risk conflict

In the monotonic concession protocol, determining the willingness to risk conflict is the key to the Zeuthen strategy: roughly, an agent is more willing to risk conflict if it has less to lose from conflict, and the agent that should concede is the one that is least willing to risk conflict (i.e., has the most to lose from conflict).

Winner determination problem

In combinatorial auctions, winner determination amounts to finding an allocation of goods that maximizes value – an NP-hard problem..

Winner's curse

The idea that if you win in an auction, then you probably overestimated the value of the good, since nobody else was willing to bid as much as you.

XML

The eXtensible Markup Language – a kind of cut-down version of SGML, allowing web users to define new tags for their applications in a structured way.

XOR bids

A type of bidding language for combinatorial auctions, in which a valuation function is defined as the ‘exclusive or’ of a number of atomic bids.

Yes/no game

A voting scenario in which an electorate must decide between two outcomes; for example, continue with the status quo or adopt a new law.

Zero sum

A game is zero sum if for every outcome, the sum of utilities from that outcome comes to 0: if we are playing a zero sum game, then the best outcome for me is the worst outcome for you, and for me to get positive utility, you must get negative utility, and vice versa.

Zeuthen strategy

A Nash equilibrium strategy for bargaining in the monotonic concession protocol: roughly, it says that the agent who should concede next should be the one that is least willing to risk conflict.

References

- Adler, M. R., Davis, A. B., Weihmayer, R., and Worrest, R. W. (1989). Conflict resolution strategies for nonhierarchical distributed agents. In Gasser, L. and Huhns, M., editors, *Distributed Artificial Intelligence Volume II*, pages 139–162. Pitman Publishing: London and Morgan Kaufmann: San Mateo, CA.
- Agha, G. (1986). *ACTORS: A Model of Concurrent Computation in Distributed Systems*. The MIT Press: Cambridge, MA.
- Agha, G., Wegner, P., and Yonezawa, A., editors (1993). *Research Directions in Concurrent Object-Oriented Programming*. The MIT Press: Cambridge, MA.
- Agre, P. and Chapman, D. (1987). PENG!: An implementation of a theory of activity. In *Proceedings of the Sixth National Conference on Artificial Intelligence (AAAI-87)*, pages 268–272, Seattle, WA.
- Agre, P. E. and Rosenschein, S. J., editors (1996). *Computational Theories of Interaction and Agency*. The MIT Press: Cambridge, MA.
- Allen, J. F. (1984). Towards a general theory of action and time. *Artificial Intelligence*, 23(2):123–154.
- Allen, J. F., Hendler, J., and Tate, A., editors (1990). *Readings in Planning*. Morgan Kaufmann Publishers: San Mateo, CA.
- Allen, J. F., Kautz, H., Pelavin, R., and Tenenbergs, J. (1991). *Reasoning About Plans*. Morgan Kaufmann Publishers: San Mateo, CA.
- Alur, R., Henzinger, T. A., and Kupferman, O. (1997). Alternating-time temporal logic. In *Proceedings of the 38th IEEE Symposium on Foundations of Computer Science*, pages 100–109, Florida.
- Amgoud, L. (1999). *Contribution a l'integration des préférences dans le raisonnement argumentatif*. PhD thesis, l'Université Paul Sabatier, Toulouse. (In French).
- Amgoud, L. and Cayrol, C. (1998). On the acceptability of arguments in preference-based argumentation. In Cooper, G. F. and Moral, S., editors, *Proceedings of the 14th Conference on Uncertainty in Artificial Intelligence*, pages 1–7, San Francisco, CA. Morgan Kaufman.
- Amgoud, L., Maudet, N., and Parsons, S. (2000). Modelling dialogues using argumentation. In *Proceedings of the Fourth International Conference on Multi-Agent Systems (ICMAS-2000)*, pages 31–38, Boston, MA.
- Andersson, A., Tenhunen, M., and Ygge, F. (2000). Integer programming for combinatorial auction winner determination. In *Proceedings of the Fourth International Conference on Multi-Agent Systems (ICMAS-2000)*, Boston, MA.
- Antoniou, G. and Harmelen, F. (2004). *A Semantic Web Primer*. The MIT Press: Cambridge, MA.
- Appelt, D. E. (1982). Planning natural language utterances. In *Proceedings of the Second National Conference on Artificial Intelligence (AAAI-82)*, pages 59–62, Pittsburgh, PA.

Appelt, D. E. (1985). *Planning English Sentences*. Cambridge University Press: Cambridge, England.

Arkin, R. C. (1998). *Behavior-Based Robotics*. The MIT Press: Cambridge, MA.

Arrow, K. J. (1951). *Social Choice and Individual Values*. Yale University Press: New Haven, CT.

Arrow, K. J., Sen, A. K., and Suzumura, K., editors (2002). *Handbook of Social Choice and Welfare Volume 1*. Elsevier Science Publishers B.V.: Amsterdam, The Netherlands.

Ausiello, G., Crescenzi, P., Gambosi, G., Kann, V., Marchetti-Spaccamela, A., and Protasi, M. (1999). *Complexity and Approximation*. Springer-Verlag: Berlin, Germany.

Austin, J. L. (1962). *How to Do Things With Words*. Oxford University Press: Oxford, England.

Axelrod, R. (1984). *The Evolution of Cooperation*. Basic Books: New York, NY.

Axelrod, R. (1997). *The Complexity of Cooperation*. Princeton University Press: Princeton, NJ.

Baader, F., Calvanese, D., McGuinness, D., Nardi, D., and Patel-Schneider, P., editors (2003). *The Description Logic Handbook*. Cambridge University Press: Cambridge, England.

Bachrach, Y. and Rosenschein, J. S. (2007). Computing the Banzhaf power index in network flow games. In *Proceedings of the Sixth International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS-2007)*, pages 335–341, Honolulu, Hawaii.

Baecker, R. M., editor (1993). *Readings in Groupware and Computer-Supported Cooperative Work*. Morgan Kaufmann Publishers: San Mateo, CA.

Barringer, H., Fisher, M., Gabbay, D., Gough, G., and Owens, R. (1989). METATEM: A framework for programming in temporal logic. In *REX Workshop on Stepwise Refinement of Distributed Systems: Models, Formalisms, Correctness (LNCS Volume 430)*, pages 94–129. Springer-Verlag: Berlin, Germany.

Bartholdi, J. J., Tovey, C. A., and Trick, M. A. (1989). The computational difficulty of manipulating an election. *Social Choice and Welfare*, 6:227–241.

Bates, J. (1994). The role of emotion in believable agents. *Communications of the ACM*, 37(7):122–125.

Bates, J., Bryan Loyall, A., and Scott Reilly, W. (1992a). An architecture for action, emotion, and social behaviour. Technical Report CMU-CS-92-144, School of Computer Science, Carnegie-Mellon University, Pittsburgh, PA.

Bates, J., Bryan Loyall, A., and Scott Reilly, W. (1992b). Integrating reactivity, goals, and emotion in a broad agent. Technical Report CMU-CS-92-142, School of Computer Science, Carnegie-Mellon University, Pittsburgh, PA.

Bauer, B., Müller, J. P., and Odell, J. (2001). Agent UML: A formalism for specifying

multiagent software systems. In Ciancarini, P. and Wooldridge, M., editors, *Agent-Oriented Software Engineering – Proceedings of the First International Workshop AOSE-2000 (LNCS Volume 1957)*, pages 91–104. Springer-Verlag: Berlin, Germany.

Bechhofer, S., van Harmelen, F., Hendler, J., Horrocks, I., McGuinness, D. L., Patel-Schneider, P. F., and Stein, L. A. (2004). OWL web ontology. See language reference, <http://www.w3.org/TR/owl-ref/>.

Bellifemine, F., Caire, G., and Greenwood, D. (2007). *Developing multi-agent systems with JADE*. John Wiley & Sons.

Belnap, N. (1991). Backwards and forwards in the modal logic of agency. *Philosophy and Phenomenological Research*, LI(4):777–807.

Belnap, N. and Perloff, M. (1988). Seeing to it that: a canonical form for agentives. *Theoria*, 54:175–199.

Ben-Ari, M. (1990). *Principles of Concurrent and Distributed Programming*. Prentice Hall.

Ben-Ari, M. (1993). *Mathematical Logic for Computer Science*. Prentice Hall.

Bench-Capon, T. J. M. (2003). Persuasion in practical argument using value-based argumentation frameworks. *Journal of Logic and Computation*, 13(3):429–448.

Bench-Capon, T. J. M. and Sartor, G. (2003). A model of legal reasoning with cases incorporating theories and values. *Artificial Intelligence*, 150(1-2):97–143.

Bench-Capon, T. J. M., Doutre, S., and Dunne, P. E. (2007). Audiences in argumentation frameworks. *Artificial Intelligence*, 171(1):42–71.

Bench-Capon, T. J. M. and Dunne, P. E. (2007). Argumentation in artificial intelligence. *Artificial Intelligence*, 171:619–641.

Benerecetti, M., Giunchiglia, F., and Serafini, L. (1999). A model checking algorithm for multiagent systems. In Müller, J. P., Singh, M. P., and Rao, A. S., editors, *Intelligent Agents V (LNAI Volume 1555)*. Springer-Verlag: Berlin, Germany.

Bergenti, F., Gleizes, M.-P., and Zambonelli, F., editors (2004). *Methodologies and Software Engineering for Agent Systems*. Kluwer Academic Publishers: Dordrecht, The Netherlands.

Berners-Lee, T. (1999). *Weaving the Web*. Orion Business: London.

Berners-Lee, T., Hendler, J., and Lassila, O. (2001). The semantic web. *Scientific American*, 284(5):34–43.

Besnard, P. and Hunter, A. (2008). *Elements of Argumentation*. The MIT Press: Cambridge, MA.

Bilbao, J. M., Fernández, J. R., Jiminéz, N., and López, J. J. (2002). Voting power in the European Union enlargement. *European Journal of Operational Research*, 143:181–196.

Binmore, K. (1992). *Fun and Games: A Text on Game Theory*. D. C. Heath and Company: Lexington, MA.